Prepare the lab file (lab_file.pdf) with the following content:

1. Introduction:

1. What is Git?

Git is a free, open-source distributed version control system that helps developers track changes, collaborate on code, and manage multiple versions of projects efficiently. It was originally created by Linus Torvalds in 2005 and is now the most widely used version control tool worldwide.

Define Git as a version control system

Git is a distributed version control system (VCS) that records changes to files, especially source code, enabling developers to track history, collaborate, and revert to earlier versions when needed. It is the most widely used VCS in modern software development

Explain its purpose (tracking changes, collaboration, etc.)

Tracking Changes:

- Git records every modification made to files in a project.
- Developers can see who changed what, when, and why.
- It allows rolling back to earlier versions if something breaks.

Collaboration:

- Multiple developers can work on the same project without overwriting each other's work.
- Git's branching and merging system lets teams experiment independently and then integrate their work.
- It supports distributed workflows, meaning everyone has a full copy of the project history.

Reliability & Redundancy:

- Because Git is distributed, every developer's machine contains a full copy of the repository.
- This eliminates a single point of failure— if the central server goes down, the project is still safe.

2. What is Version Control?

Version control is a system that manages changes to files over time, allowing you to record, track, and organize modifications. It's most commonly used in software development to handle source code, but it can apply to any set of files that evolve over time.

Explain the concept of version control

The concept of version control is about managing the evolution of files and projects over time. Think of it as a system that acts like a history keeper and collaboration manager for your work.

Core Idea:

Version control records every change made to a file (or set of files), so you can:

- Track history: See what changed, when, and by whom.
- Restore past versions: Roll back if something breaks.
- Work together: Multiple people can edit the same project without overwriting each other's work.

How It Works:

- Repository: A storage space where all files and their history live.
- Commits: Snapshots of the project at a given point in time.
- Branches: Parallel lines of development where new features or experiments can happen safely.
- Merging: Combining changes from different branches into one unified version

Importance in software development

The importance of version control in software development is enormous.

Preserving Project History:

- Every commit acts like a snapshot of the project at that moment.
- This history is invaluable for auditing, learning from past decisions, and understanding how the project evolved.
- You can roll back to earlier versions if something breaks.

Supporting Experimentation:

Developers can create branches to test new ideas without affecting the main codebase.

If the experiment fails, the branch can be discarded safely.

If successful, it can be merged back into the main project

3. What is GitHub?

GitHub is a cloud-based platform built on top of Git that hosts repositories, enables collaboration, and provides tools for managing software projects. It's widely used by developers to share code, track changes, and work together remotely.

Define GitHub as a platform for hosting repositories

GitHub provides a centralized online space where developers can upload their Git repositories.

It builds on Git's version control system by adding collaboration tools, project management features, and community sharing

Mention its role in collaboration

Key Roles in Collaboration:

Shared Repository Hosting: Teams can store their code in one central place, accessible to all members.

Pull Requests: Developers propose changes, and teammates can review, discuss, and approve them before merging into the main project.

Code Reviews: Built-in review tools allow comments, suggestions, and approvals, ensuring quality and consistency.

Issue Tracking: Teams can log bugs, feature requests, and tasks, keeping everyone aligned on priorities.

4. Difference Between Git and GitHub .Write at least 3–5 clear differences .

Aspect	Git	GitHub
Definition	A distributed version control system (VCS) used to track changes in code.	A cloud-based platform that hosts Git repositories and adds collaboration tools.

Functionality	Manages code history, branching, merging, and version tracking locally.	Provides online repository hosting, pull requests, issue tracking, and project management.
Usage	Works offline; developers can commit and manage code on their own machines.	Requires internet access to share repositories and collaborate with others.
Collaboration	Collaboration requires manual sharing of repositories (e.g., via email or file transfer).	Built-in collaboration features like code reviews, discussions, and team management.
Scope	Focused purely on version control.	Extends Git with social coding, automation (GitHub Actions), and community contributions.

2. Basic Git Commands

Include a list of all Git/Git Bash & vi Editor commands performed during the previous lab.

Git Configuration Commands

```
git --version
git config --global user.name "Your Name"
git config --global user.email "email@gmail.com"
git config user.name
git config user.email
git config --list
git help
```

Navigation Commands

```
pwd
cd folder_name/location
```

File and Folder Listing Commands

```
ls
ls -l
ls -a
ls -d */
```

File Operations

touch filename

touch file1 file2 file3

rm filename

Folder Operations

mkdir folder_name

mkdir folder1 folder2 folder3

rmdir folder_name

rm -r foldername

Viewing File Content

cat filename

Writing Text to a File

echo "text-content" > filename

echo "text" >> filename

Writing Text Using cat

cat > filename

cat >> filename

Copy Content Between Files

cat file1.txt > file2.txt

Editing with Nano

nano file.txt

- Save: Ctrl+O
- Exit: Ctrl+X

Editing with Vi/Vi

git var GIT_EDITOR

git config --global core.editor "vi"

vi filename.txt

Insert Mode commands:

- o i
- o I
- o a
- o A
- o o
- o O
- o Esc

Save & Quit in Vi

:w

:q

:wq or :x

:q!

Delete Commands in Vi

x

dd

d$

dθ

Copy & Paste in Vi

y

p

P

For every command, write an explanation of what it does.

Git Configuration

git --version → Shows the installed Git version.

git config --global user.name "Your Name" → Sets your Git username globally (applies to all repositories).

git config --global user.email "email@gmail.com" → Sets your Git email globally.

git config user.name → Displays the current username setting.

git config user.email → Displays the current email setting.

git config --list → Lists all Git configuration settings.

git help → Opens Git help documentation.

Navigation Commands

pwd → Prints the current working directory path.

cd folder_name/location → Changes directory to the specified folder.

File and Folder Listing

ls -l → Lists files with detailed info (permissions, size, date).

ls -a → Lists all files including hidden ones.

ls -d */ → Lists only directories.

ls → Lists files and folders in the current directory.

File Operations

touch filename → Creates an empty file.

touch file1 file2 file3 → Creates multiple empty files at once.

rm filename → Deletes a file.

Folder Operations

mkdir folder_name → Creates a new folder.

mkdir folder1 folder2 folder3 → Creates multiple folders at once.

rmdir folder_name → Removes an empty folder.

rm -r foldername → Removes a folder and its contents recursively.

Viewing File Content

cat filename → Displays the contents of a file.

Writing Text to a File

echo "text-content" > filename → Writes text to a file (overwrites if it exists).

echo "text" >> filename → Appends text to a file without deleting existing content.

Writing Text Using cat

cat > filename → Creates a file and lets you type content directly (save with Ctrl+D).

cat >> filename → Appends typed content to an existing file (save with Ctrl+D).

Copy Content Between Files

cat file1.txt > file2.txt → Copies content of file1 into file2 (overwrites file2).

Editing with Nano

nano file.txt → Opens Nano editor for editing a file.

- Ctrl+O → Save changes.
- Ctrl+X → Exit Nano.

Editing with Vi/Vim

git var GIT_EDITOR → Checks default editor.

git config --global core.editor "vi" → Sets Vi as default editor.

vi filename.txt → Opens file in Vi editor.

Insert Mode commands:

i → Insert before cursor

I → Insert at beginning of line

a → Append after cursor

A → Append at end of line

o → New line below cursor

O → New line above cursor

Esc → Exit insert mode

Save & Quit in Vi

:w → Save file

:q → Quit editor

:wq or :x → Save and quit

:q! → Quit without saving

Delete Commands in Vi

x → Delete character

dd → Delete current line

d$ → Delete from cursor to end of line

dθ → Delete from cursor to start of line

Copy & Paste in Vi

y → Yank (copy)

p → Paste after cursor

P → Paste before cursor

After each command and its description, attach a relevant screenshot showing: Command execution in terminal (Git Bash) Output/result of the command

Git Configuration Commands

git --version → Shows the installed Git version.

```
Rakshithasai@Rakshitha MINGW64 ~
$ git --version
git version 2.53.0.windows.1
```

git config --global user.name "Your Name" → Sets your name globally for all repositories.

```
Rakshithasai@Rakshitha MINGW64 ~
$ git config --global user.name "K.Rakshitha Sai"
```

git config --global user.email "email@gmail.com" → Sets your email globally for all repositories.

```
Rakshithasai@Rakshitha MINGW64 ~
$ git config --global user.email "rakshithasai2406@gmail.com"
```

git config user.name → Displays the username set for the current repository.

```
Rakshithasai@Rakshitha MINGW64 ~
$ git config user.name
K.Rakshitha Sai
```

git config user.email → Displays the email set for the current repository.

```
Rakshithasai@Rakshitha MINGW64 ~
$ git config user.email
rakshithasai2406@gmail.com
```

git config --list → Lists all Git configuration settings.

```
Rakshithasai@Rakshitha MINGW64 ~
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=K.Rakshitha Sai
user.email=rakshithasai2406@gmail.com
core.editor=vi
```

git help → Opens Git help documentation.

```
Rakshithasai@Rakshitha MINGW64 ~
$ git help
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
           [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
           [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
           [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
           [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
           <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
   clone      Clone a repository into a new directory
   init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
   add        Add file contents to the index
   mv         Move or rename a file, a directory, or a symlink
   restore    Restore working tree files
   rm         Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
   bisect     Use binary search to find the commit that introduced a bug
   diff       Show changes between commits, commit and working tree, etc
   grep       Print lines matching a pattern
   log        Show commit logs
   show       Show various types of objects
   status     Show the working tree status

grow, mark and tweak your common history
   backfill   Download missing objects in a partial clone
   branch     List, create, or delete branches
   commit     Record changes to the repository
   merge      Join two or more development histories together
   rebase     Reapply commits on top of another base tip
   reset      Set `HEAD` or the index to a known state
   switch     Switch branches
   tag        Create, list, delete or verify tags

collaborate (see also: git help workflows)
   fetch      Download objects and refs from another repository
   pull       Fetch from and integrate with another repository or a local branch
   push       Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.
```

Navigation Commands

pwd → Prints the current working directory path.

```
Rakshithasai@Rakshitha MINGW64 ~
$ pwd
/c/Users/Rakshithasai
```

cd folder_name/location → Changes directory to the specified folder.

```
Rakshithasai@Rakshitha MINGW64 ~
$ cd OneDrive
```

cd folder_name/location → Changes directory to the specified

folder.

```
Rakshithasai@Rakshitha MINGW64 ~
$ pwd
/c/Users/Rakshithasai

Rakshithasai@Rakshitha MINGW64 ~
$ cd OneDrive

Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ pwd
/c/Users/Rakshithasai/OneDrive
```

File and Folder Listing Commands

ls → Lists files and folders in the current director

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ ls
 Attachments/            Desktop/        Document.docx       Pictures/          'Python Module 5.pptx'  desktop.ini  fileo     filey.txt  foldery/  source.code.docx
'C naming conventions.docx'  'Document 1.docx'  'Personal Vault.lnk'*  'Pyhton Module 6.pptx'  debug.log          file.txt     filex.txt  filez     folderz/
```

ls -l → Lists files with detailed info (permissions, size, date).

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ ls -l
total 7357
drwxr-xr-x 1 Rakshithasai 197121       0 Oct 21 18:19  Attachments/
-rw-r--r-- 1 Rakshithasai 197121   20056 Mar  6 07:32 'C naming conventions.docx'
drwxr-xr-x 1 Rakshithasai 197121       0 Apr  6 20:12  Desktop/
-rw-r--r-- 1 Rakshithasai 197121   49903 Dec 15 16:21 'Document 1.docx'
-rw-r--r-- 1 Rakshithasai 197121   49487 Dec 15 16:21  Document.docx
-rwxr-xr-x 1 Rakshithasai 197121    1140 Apr  7 21:04 'Personal Vault.lnk'*
drwxr-xr-x 1 Rakshithasai 197121       0 Apr  6 20:12  Pictures/
-rw-r--r-- 1 Rakshithasai 197121 1887223 Feb  6 10:43 'Pyhton Module 6.pptx'
-rw-r--r-- 1 Rakshithasai 197121 5386041 Feb  6 10:44 'Python Module 5.pptx'
-rw-r--r-- 1 Rakshithasai 197121     103 Oct 20 17:52  desktop.ini
-rw-r--r-- 1 Rakshithasai 197121  171356 Apr  7 22:15  source.code.docx
```

ls -a → Lists all files including hidden ones.

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ ls -a
 ./   .849C9593-D756-4E56-8D6E-42412F2A707B  'C naming conventions.docx'  'Document 1.docx'  'Personal Vault.lnk'*  'Pyhton Module 6.pptx'  desktop.ini
 ../  Attachments/                           Desktop/                     Document.docx      Pictures/              'Python Module 5.pptx'  source.code.docx
```

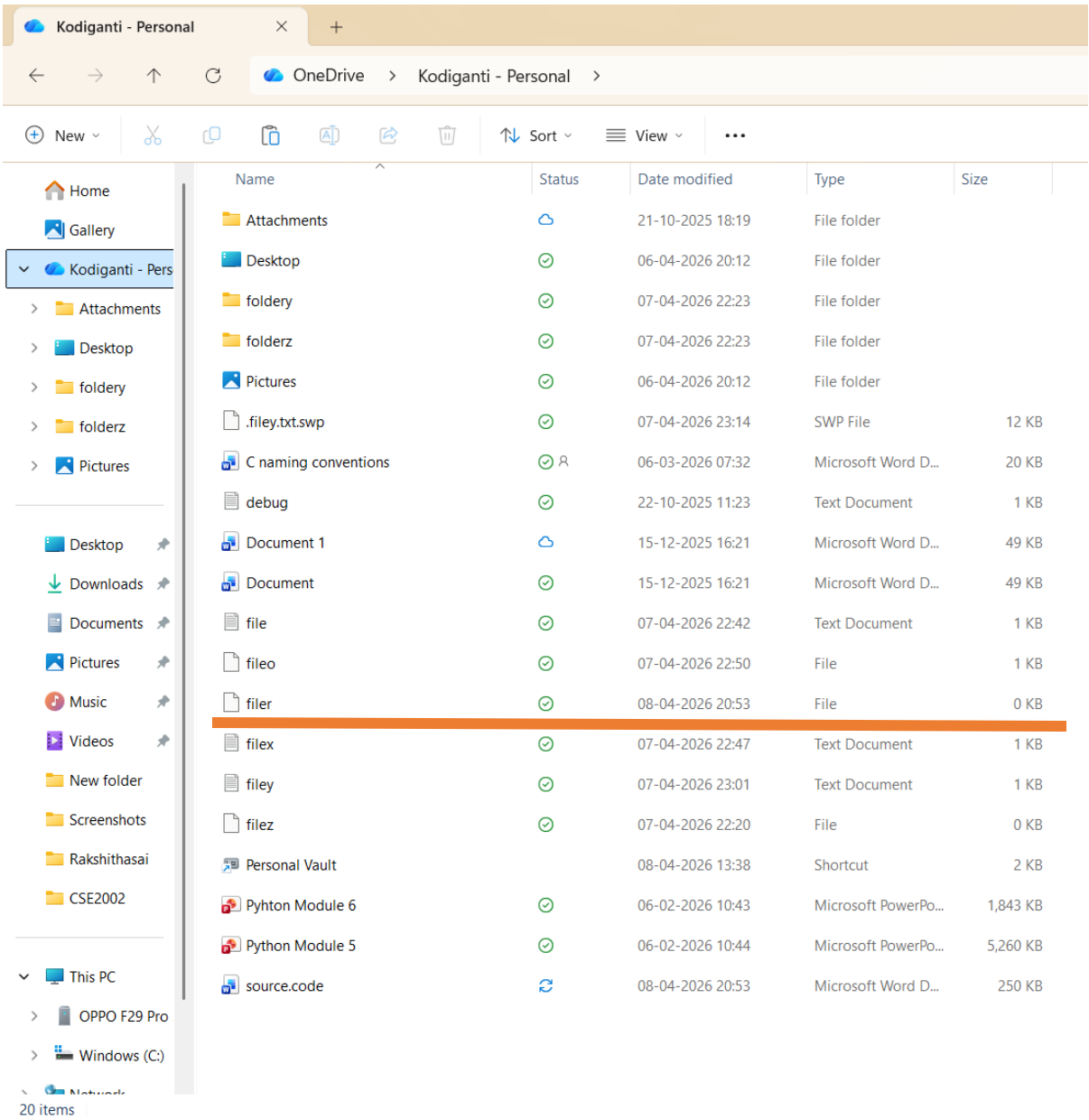
ls -d */ → Lists only directories.

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ ls -d */
Attachments//  Desktop//  Pictures//
```

File Operations

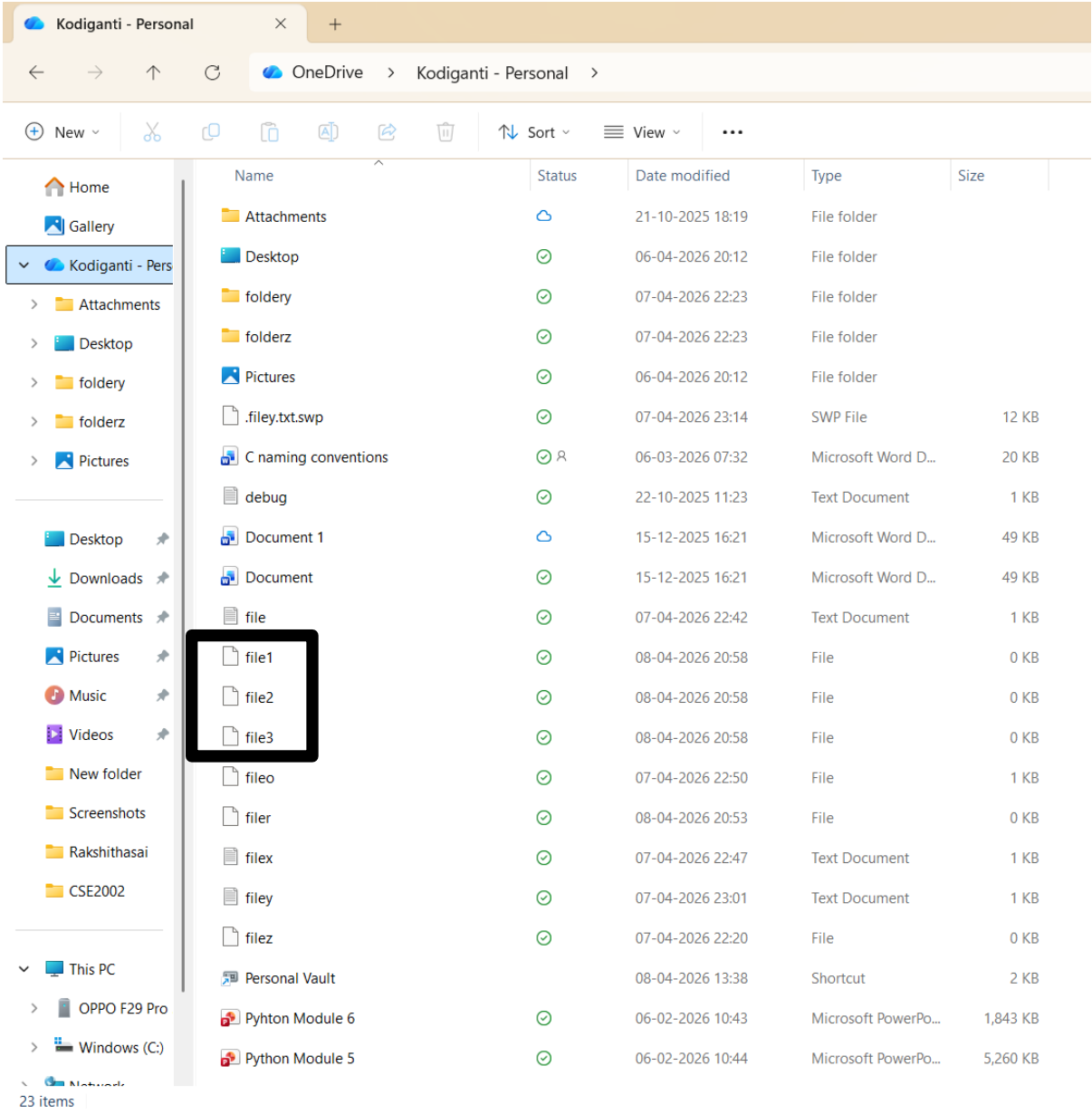
touch filename → Creates an empty file.

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ touch filer
```



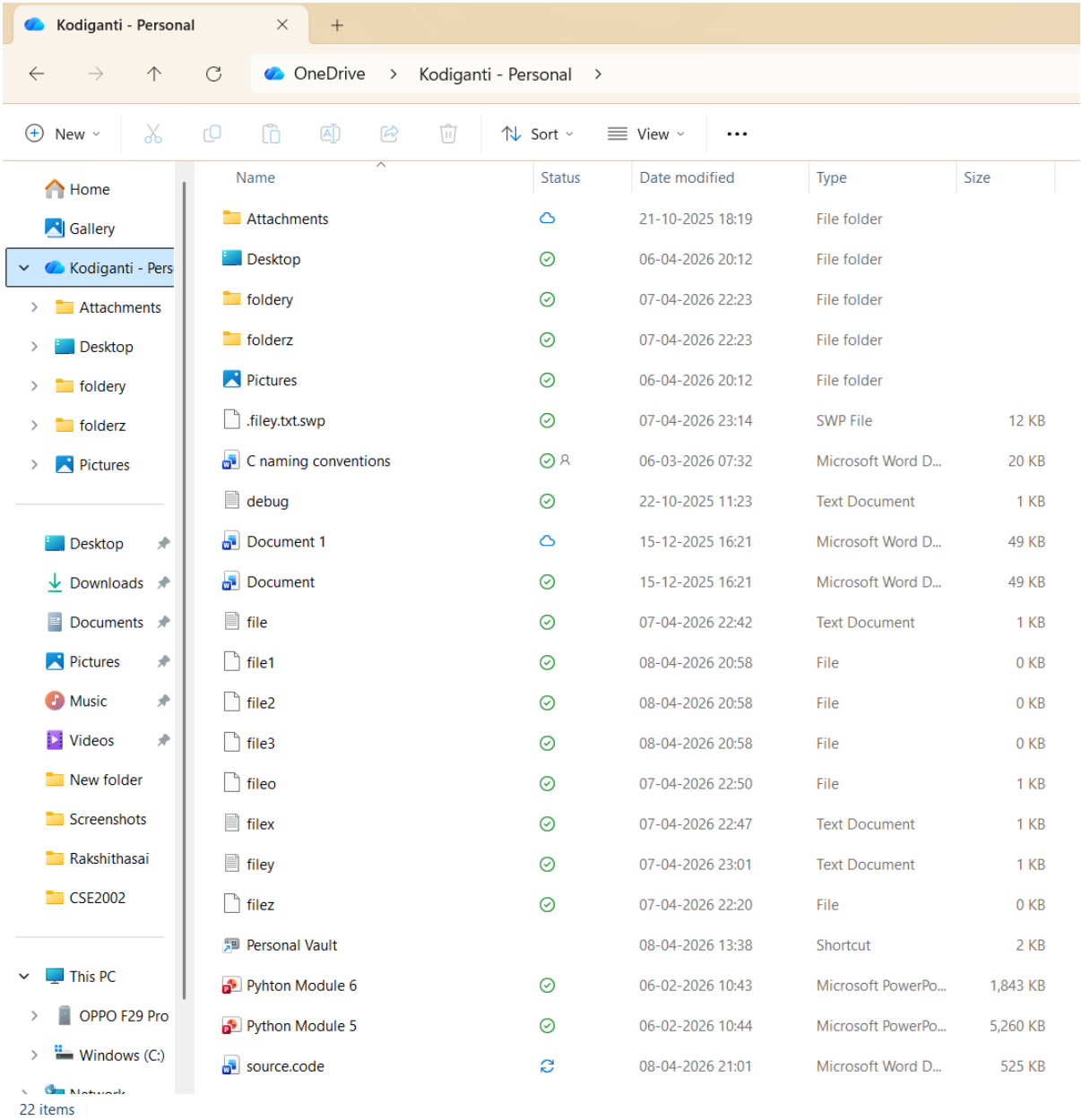
touch file1 file2 file3 → Creates multiple empty files at once.

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ touch file1 file2 file3
```



rm filename → Deletes a file.

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ rm filer
```

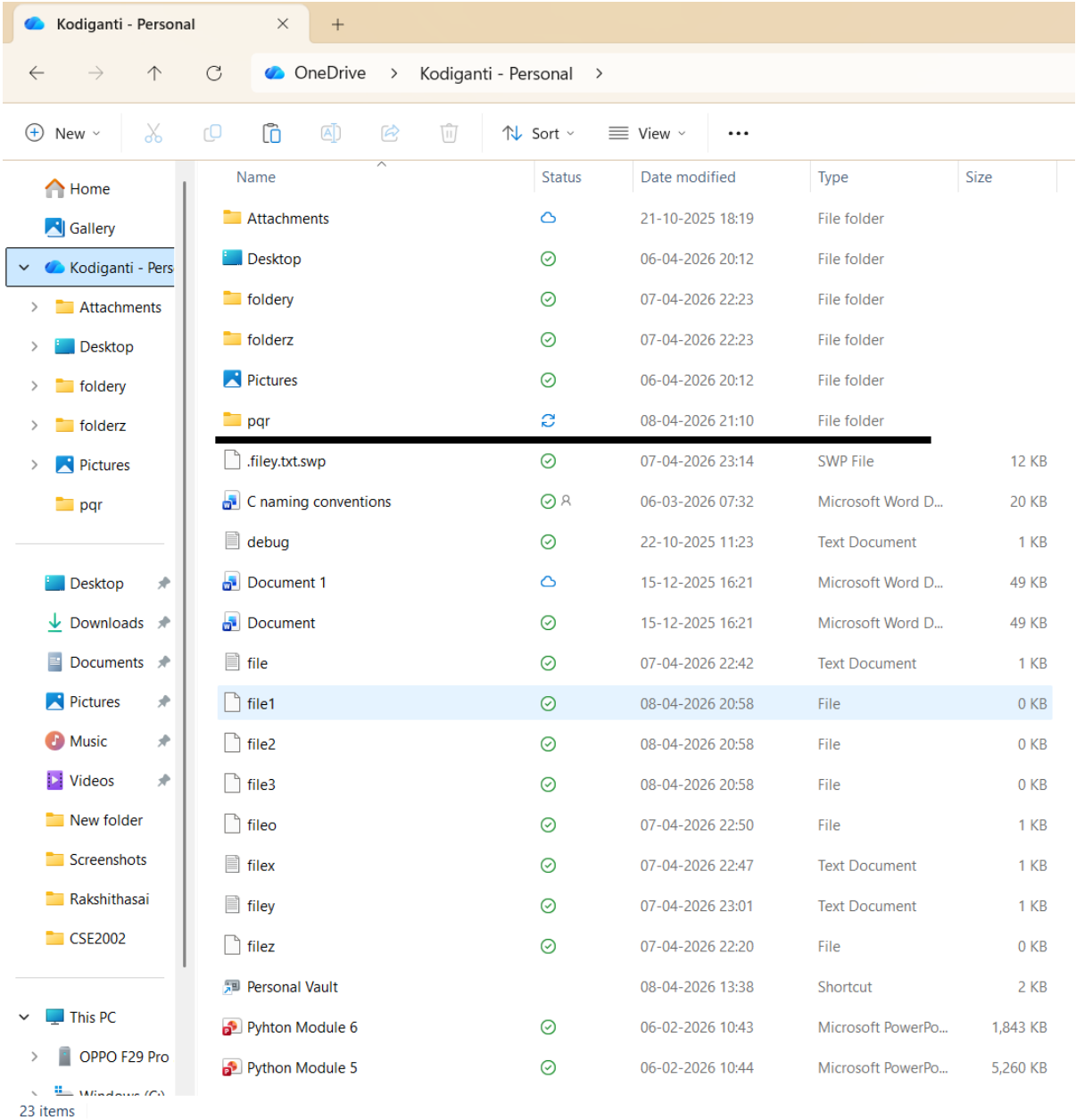


In the above image file “filer” is removed.

Folder Operations

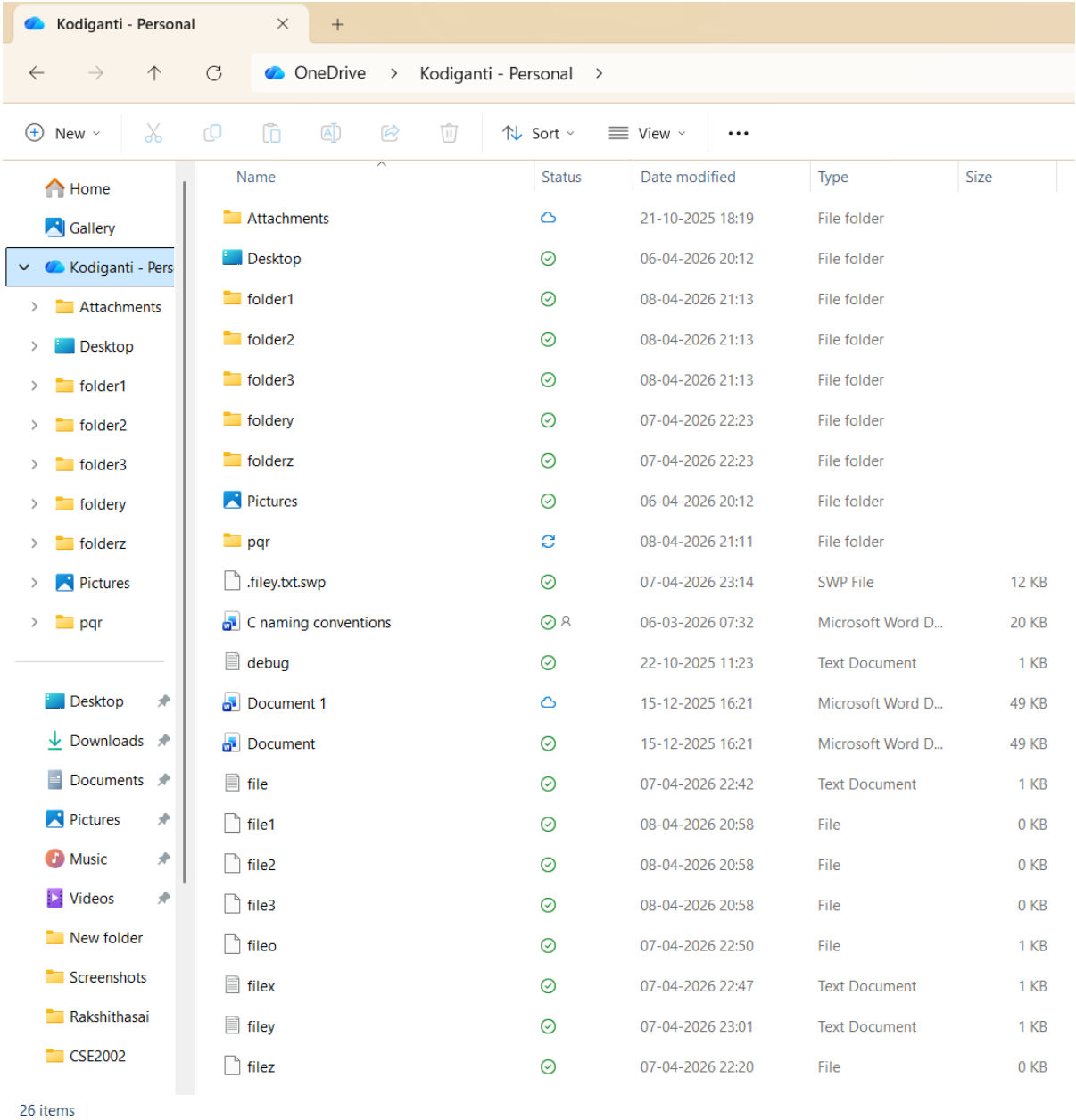
mkdir folder_name → Creates a new folder.

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ mkdir pqr
```



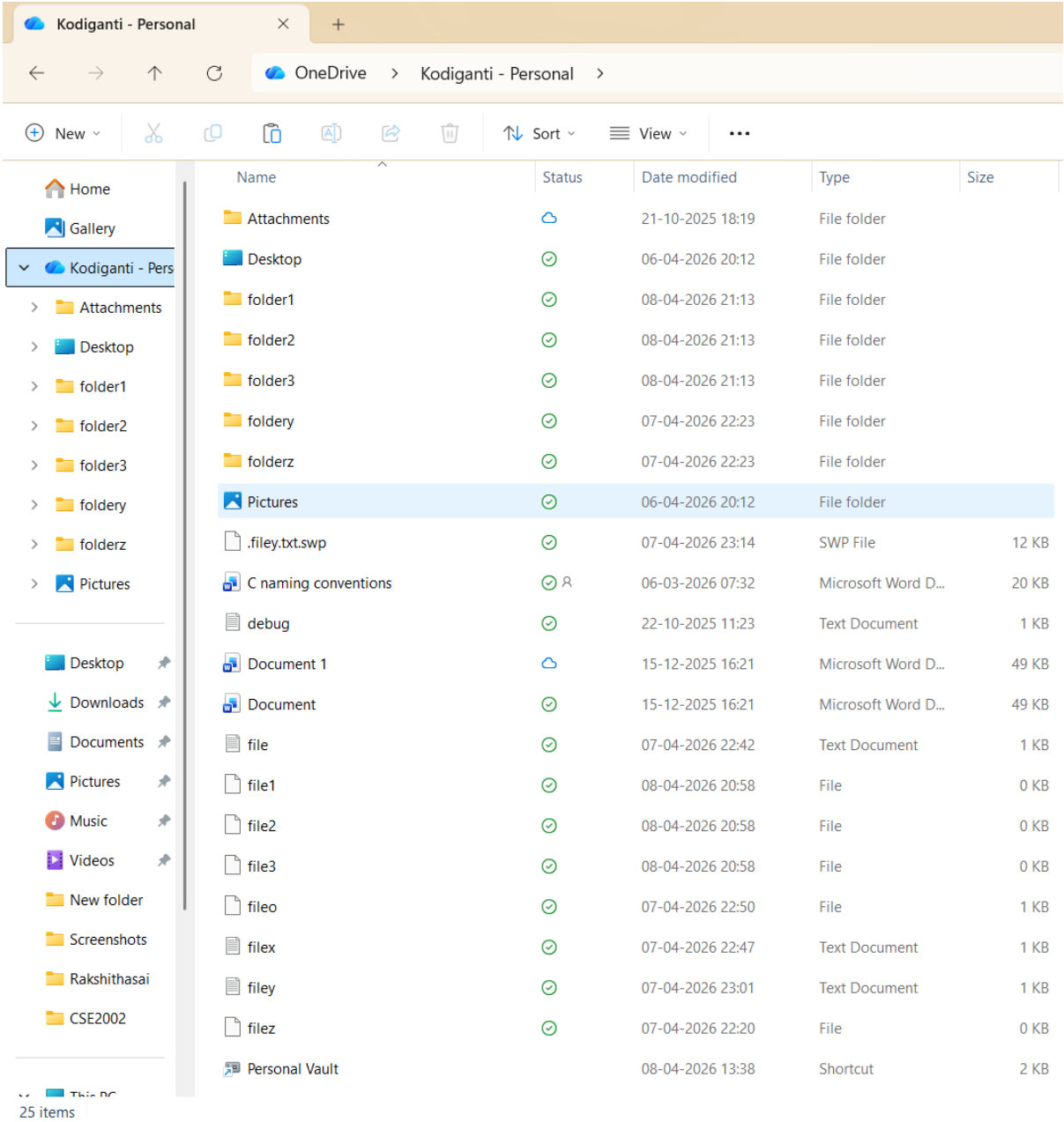
mkdir folder1 folder2 folder3 → Creates multiple folders at once.

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ mkdir folder1 folder2 folder3
```



rmdir folder_name → Removes an empty folder.

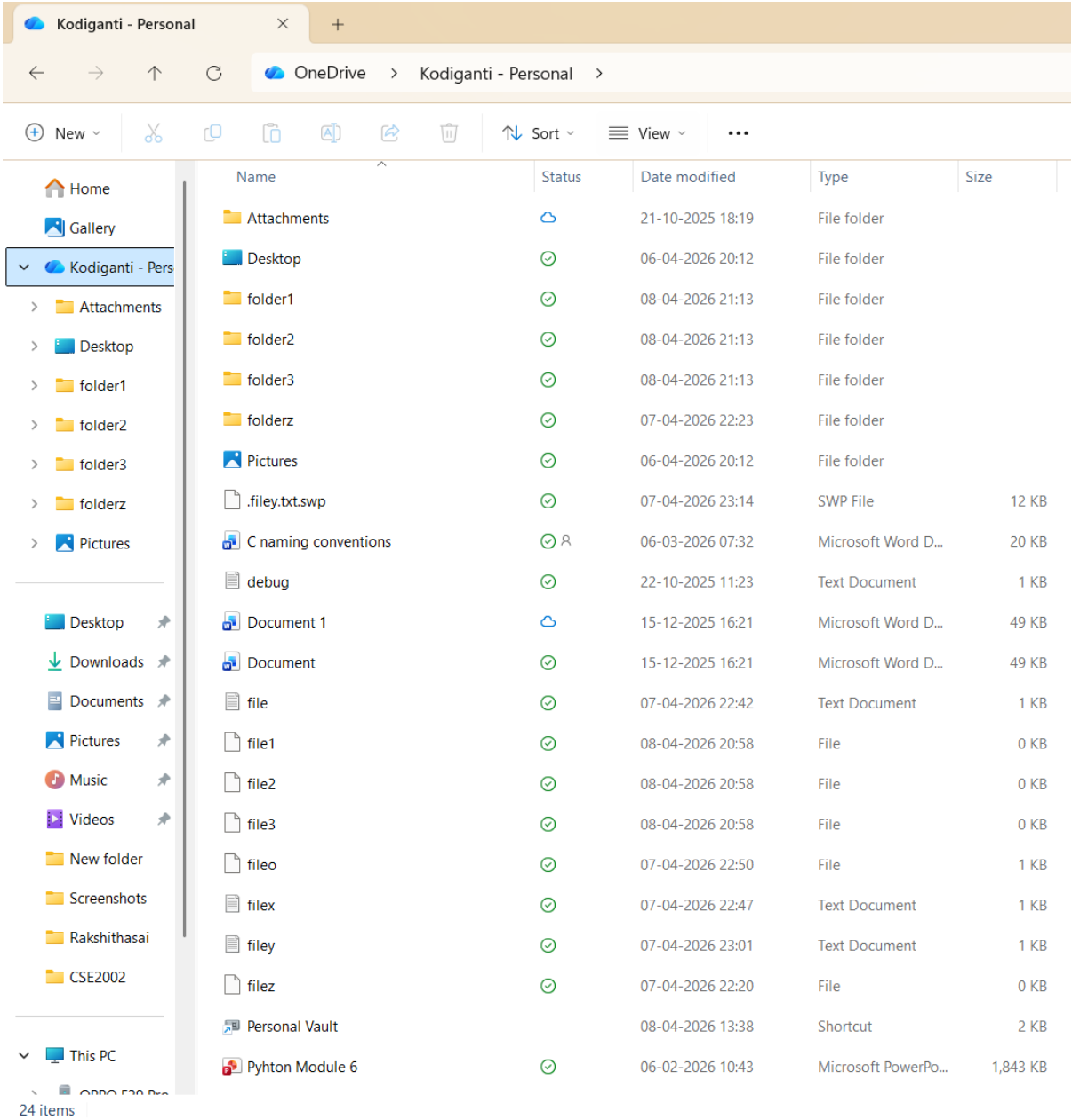
```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ rmdir pqr
```



In the above image folder ”pqr” is removed.

rm -r foldername → Removes a folder and its contents recursively.

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ rm -r foldery
```

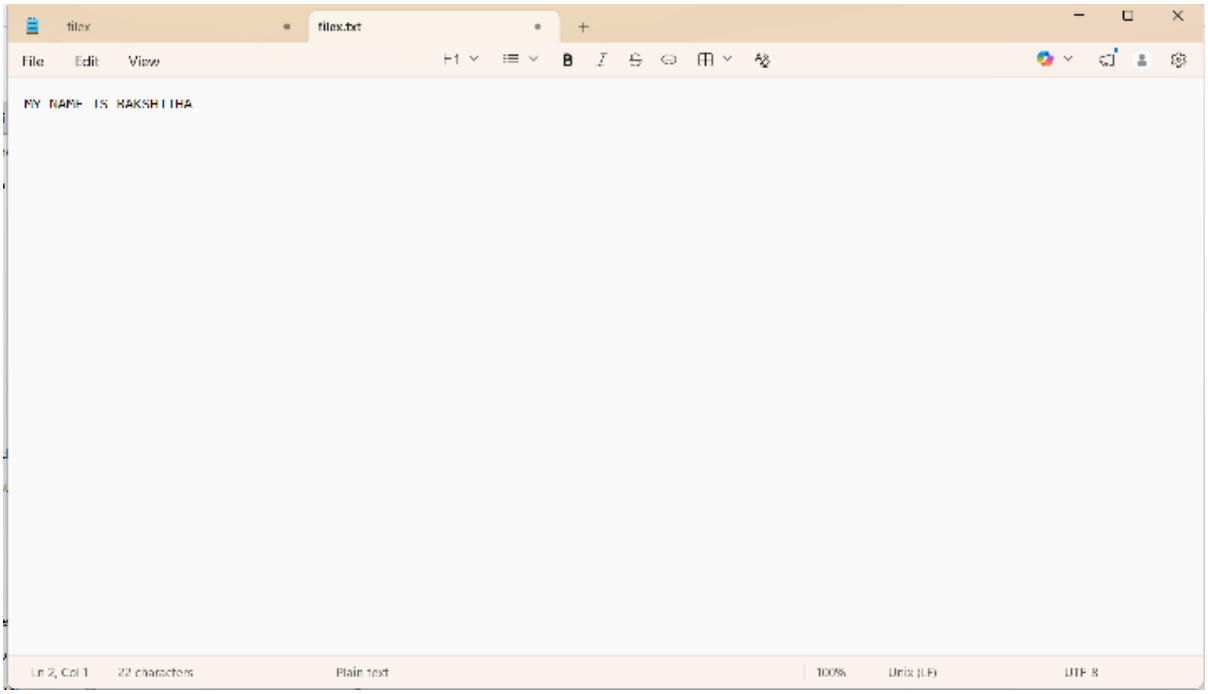


In the above image folder “foldery” is removed.

Viewing File Content

cat filename → Displays contents of a file.

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ cat filex.txt
MY NAME IS RAKSHITHA
```

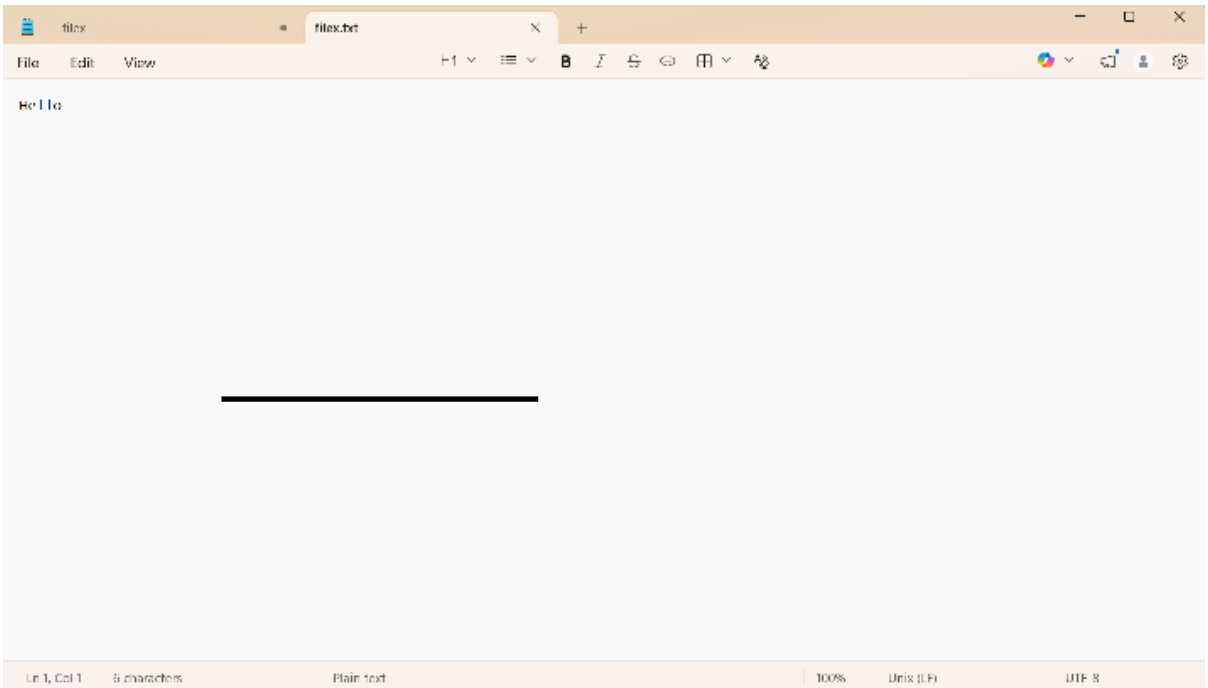


In the above image shows content of “filex”.

Writing Text to a File

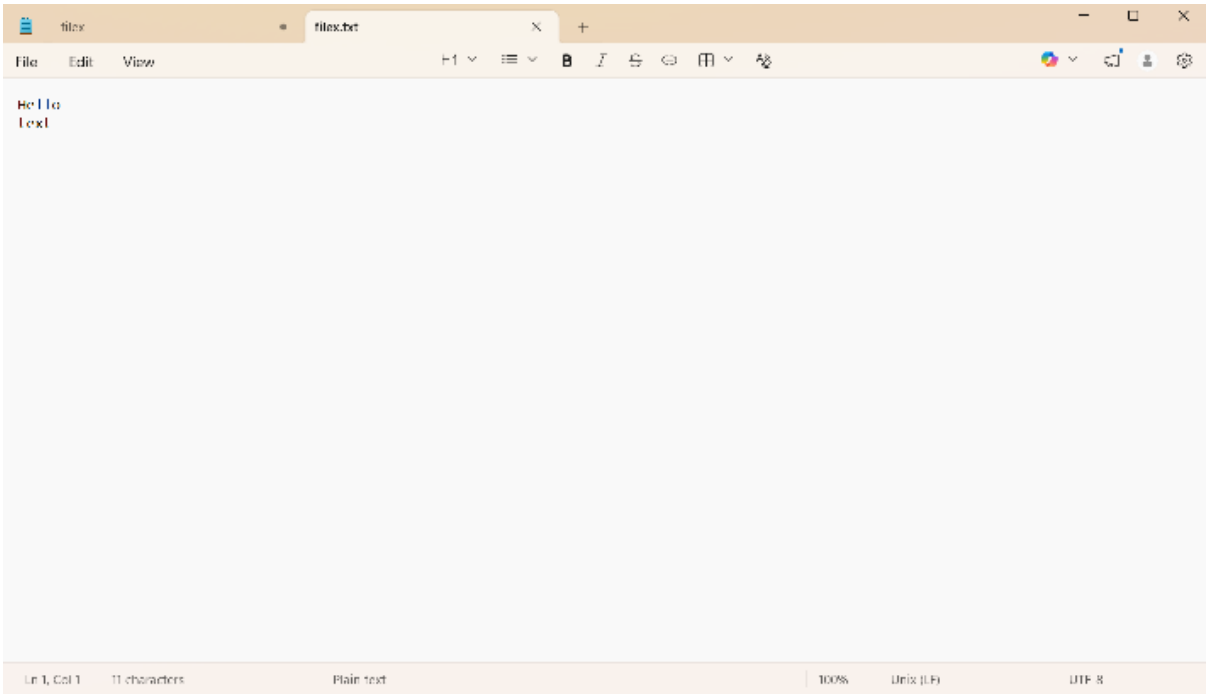
echo "text-content" > filename → Writes text to a file (overwrites if exists).

```
akshithasai@Rakshitha MINGW64 ~/OneDrive
$ echo "Hello" > filex.txt
```



echo "text" >> filename → Appends text to a file without deleting existing content.

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ echo "text" >> filex.txt
```



Writing Text Using cat

cat > filename → Creates a file and lets you type text directly (save with Ctrl+D).

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ cat > fileo
SOURCE CODE MANAGEMENT
```

Kodiganti - Personal

OneDrive > Kodiganti - Personal >

New | Sort | View

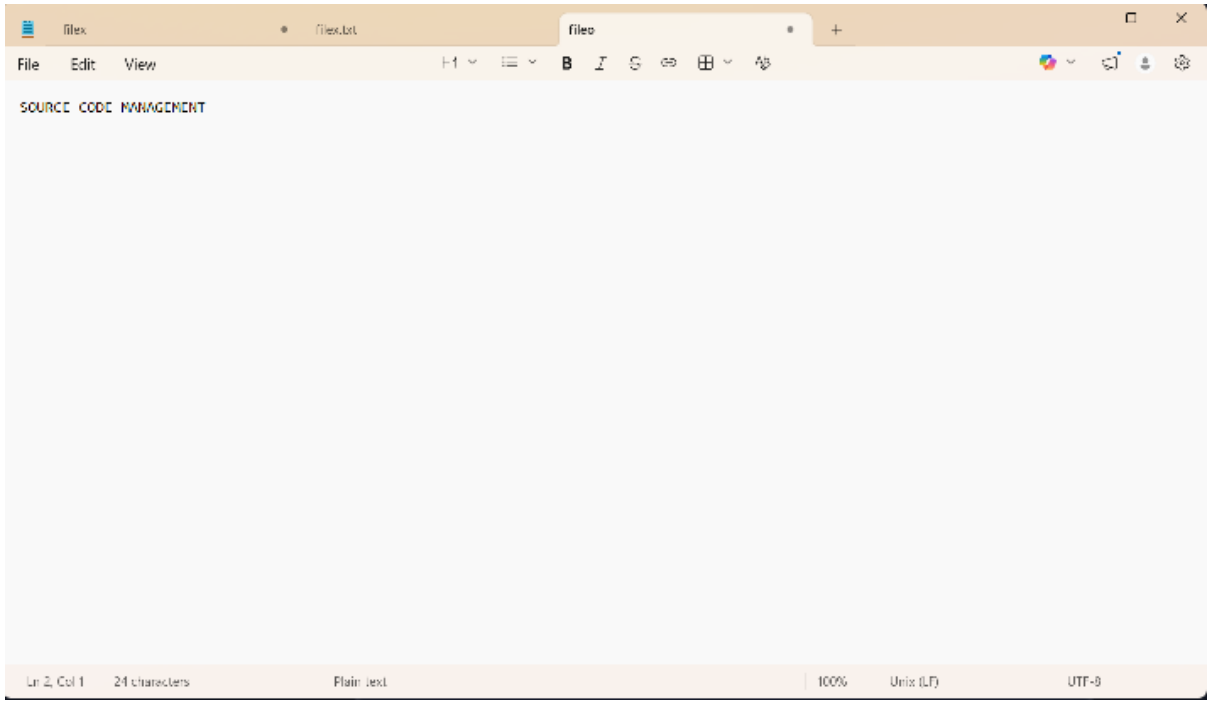
Home
Gallery
Kodiganti - Pers
Attachments
Desktop
folder1
folder2
folder3
folderz
Pictures

Desktop
Downloads
Documents
Pictures
Music
Videos
New folder
Screenshots
Rakshithasai
CSE2002

This PC

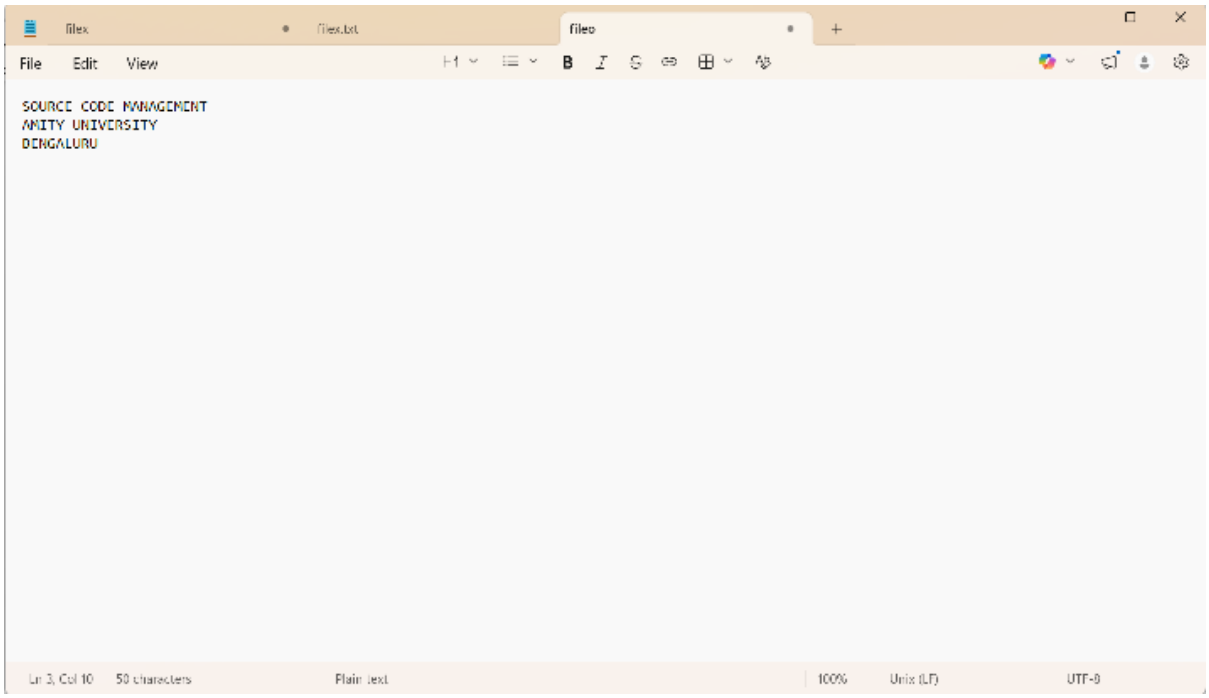
Name	Status	Date modified	Type	Size
Attachments		21-10-2025 18:19	File folder	
Desktop		06-04-2026 20:12	File folder	
folder1		08-04-2026 21:13	File folder	
folder2		08-04-2026 21:13	File folder	
folder3		08-04-2026 21:13	File folder	
folderz		07-04-2026 22:23	File folder	
Pictures		06-04-2026 20:12	File folder	
.filey.txt.swp		07-04-2026 23:14	SWP File	12 KB
C naming conventions		06-03-2026 07:32	Microsoft Word D...	20 KB
debug		22-10-2025 11:23	Text Document	1 KB
Document 1		15-12-2025 16:21	Microsoft Word D...	49 KB
Document		15-12-2025 16:21	Microsoft Word D...	49 KB
file		08-04-2026 21:28	Text Document	1 KB
file1		08-04-2026 20:58	File	0 KB
file2		08-04-2026 20:58	File	0 KB
file3		08-04-2026 20:58	File	0 KB
fileo		07-04-2026 22:50	File	1 KB
filex		08-04-2026 21:31	Text Document	1 KB
filey		07-04-2026 23:01	Text Document	1 KB
filez		07-04-2026 22:20	File	0 KB
Personal Vault		08-04-2026 13:38	Shortcut	2 KB
Pyhton Module 6		06-02-2026 10:43	Microsoft PowerPo...	1,843 KB

24 items



cat >> filename → Appends text to an existing file (save with Ctrl+D).

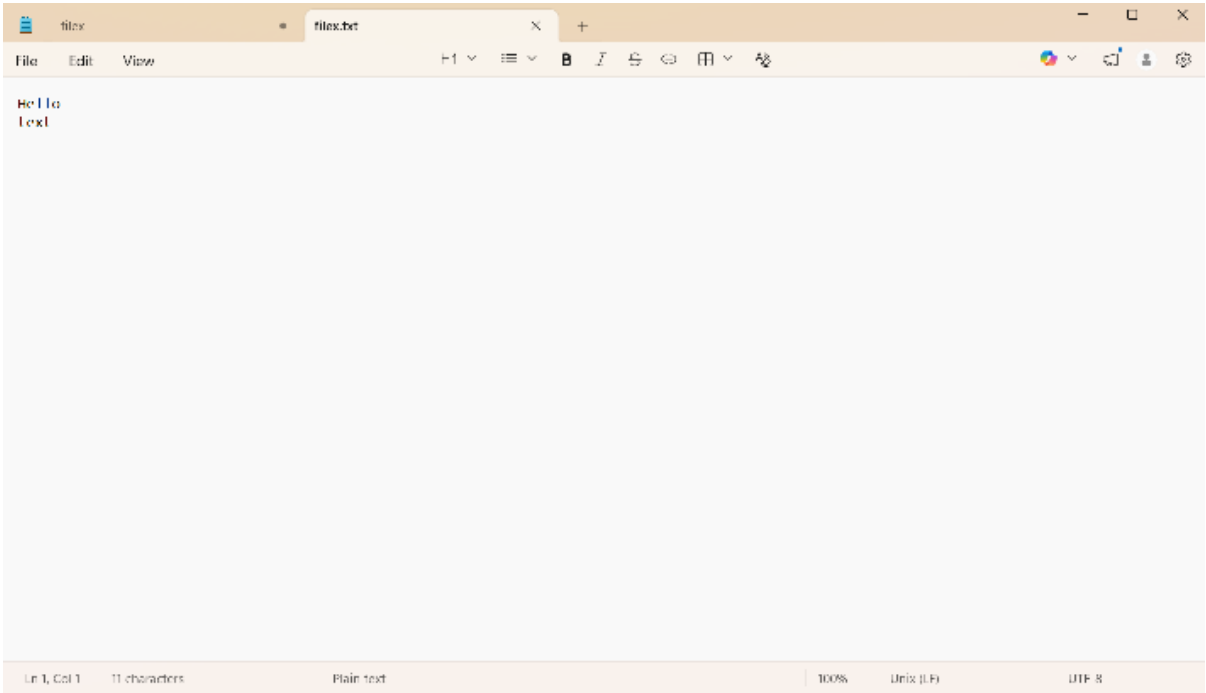
```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ cat >> fileo
AMITY UNIVERSITY
BENGALURU
```

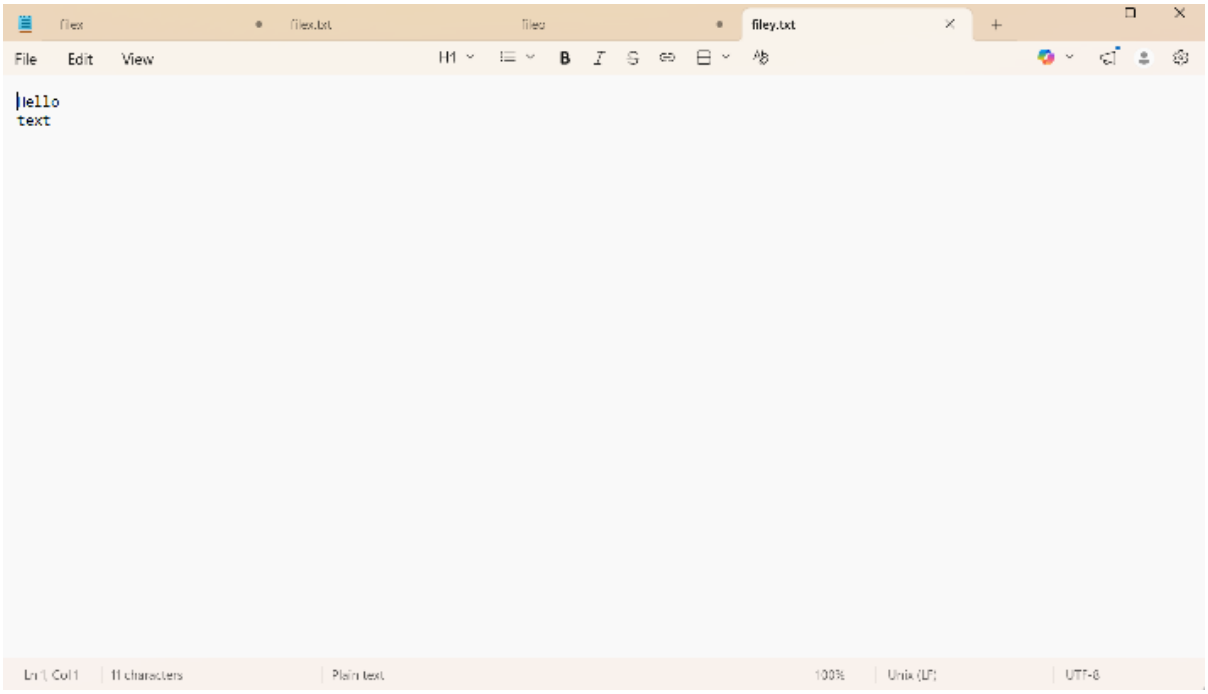


Copy Content Between Files

cat file1.txt > file2.txt → Copies content of file1 into file2.

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ cat filex.txt > fileY.txt
```





In the above two images we can observe that content of “filex.txt” is copied to “filey.txt”.

Editing with Nano

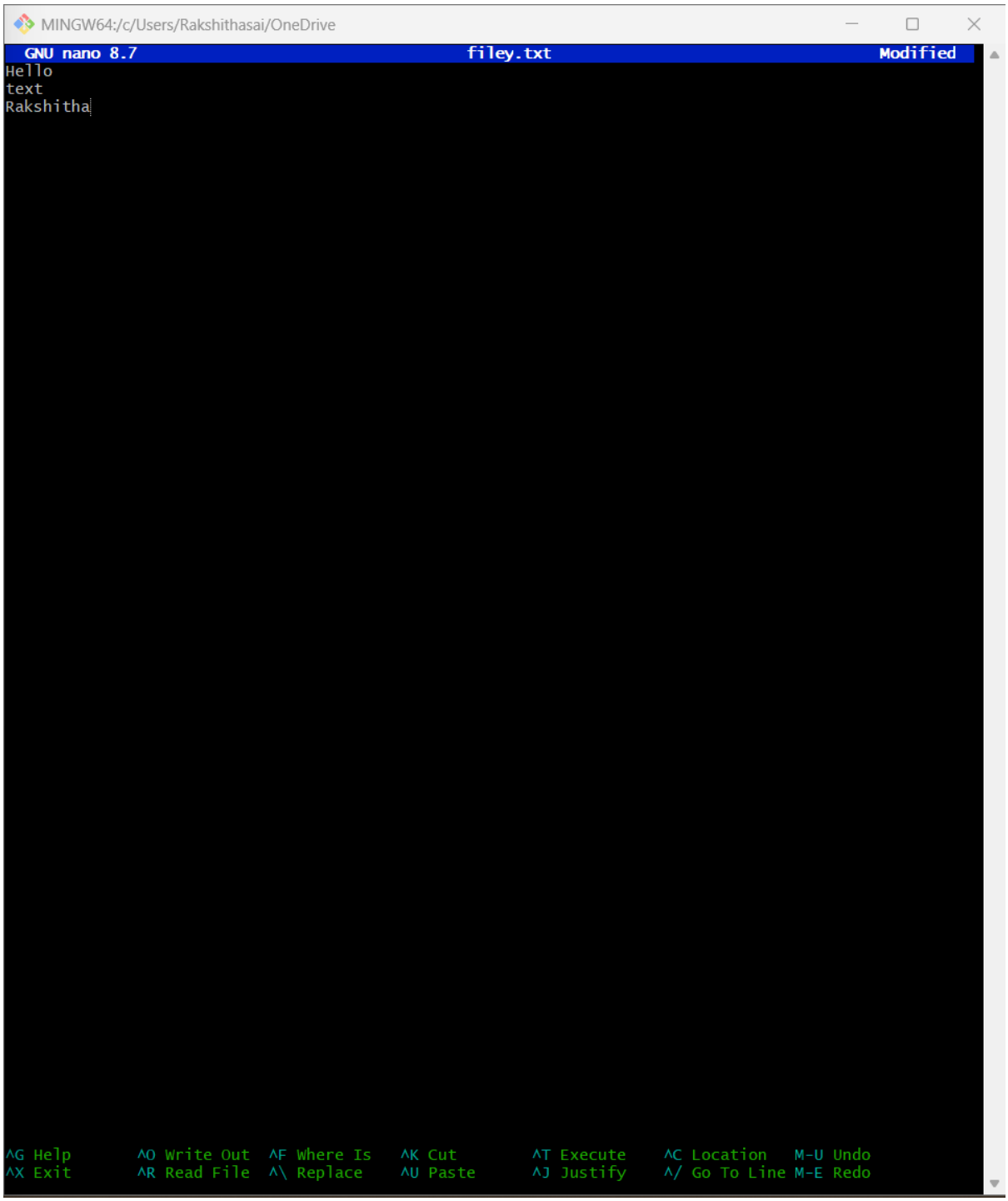
nano file.txt → Opens Nano editor.

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ nano filey.txt
```

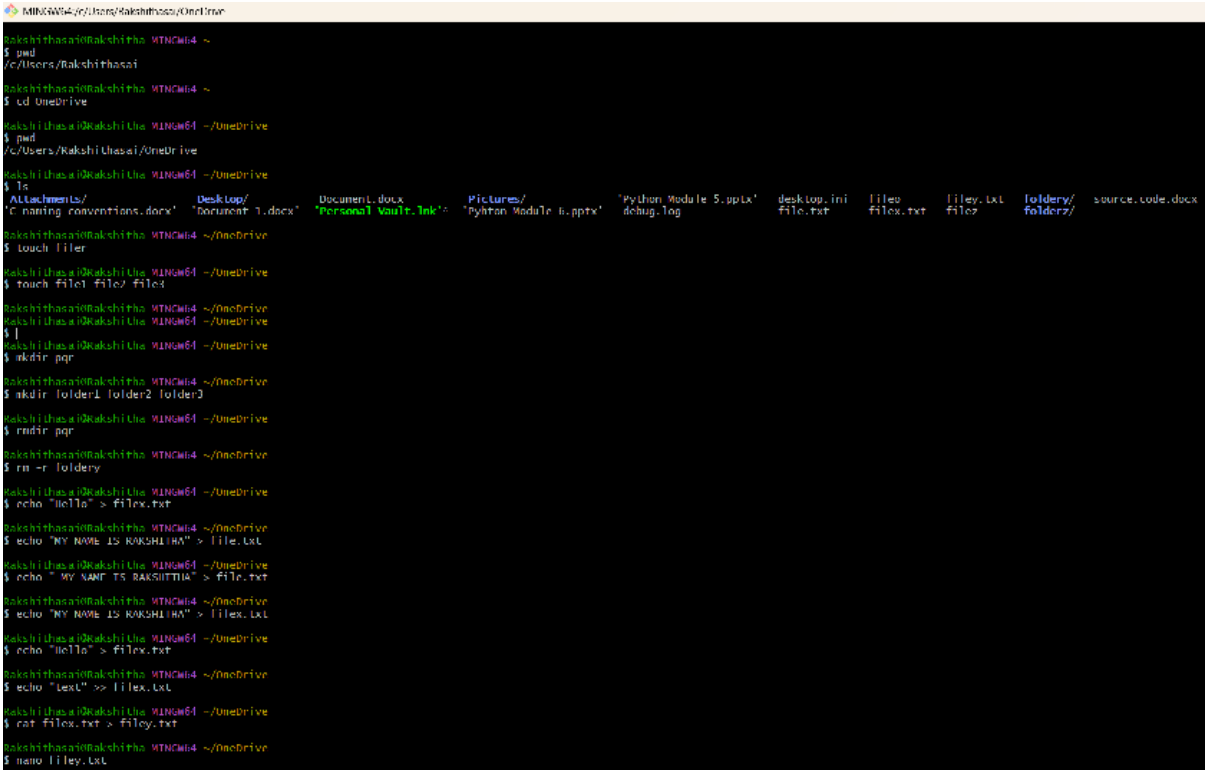
```
MINGW64:/c/Users/Rakshithasai/OneDrive
  GNU nano 8.7                    filey.txt
Hello
text

                                [ Read 2 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

- Save: Ctrl+O


MINGW64:/c/Users/Rakshithasai/OneDrive
GNU nano 8.7
filey.txt
Modified
Hello
text
Rakshitha
^G Help
^O Write Out
^F Where Is
^K Cut
^T Execute
^C Location
M-U Undo
^X Exit
^R Read File
^\ Replace
^U Paste
^J Justify
^/ Go To Line
M-E Redo

- Exit: Ctrl+X



Ctrl+X is used to return to Git Bash.In the above image Ctrl+X is used to retrun to Git bash

Editing with Vi/Vim

git var GIT_EDITOR → Checks default editor.

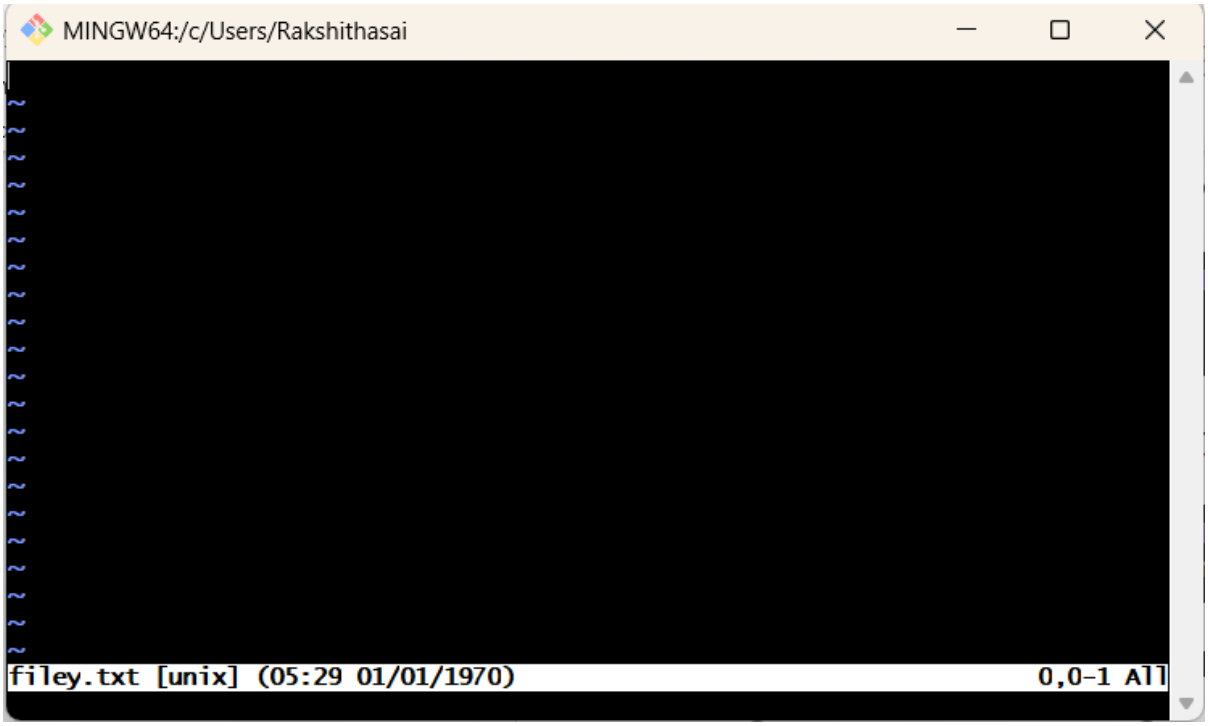
```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ git var GIT_EDITOR
vi
```

git config --global core.editor "vi" → Sets Vi as default editor.

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ git config --global core.editor "vi"
```

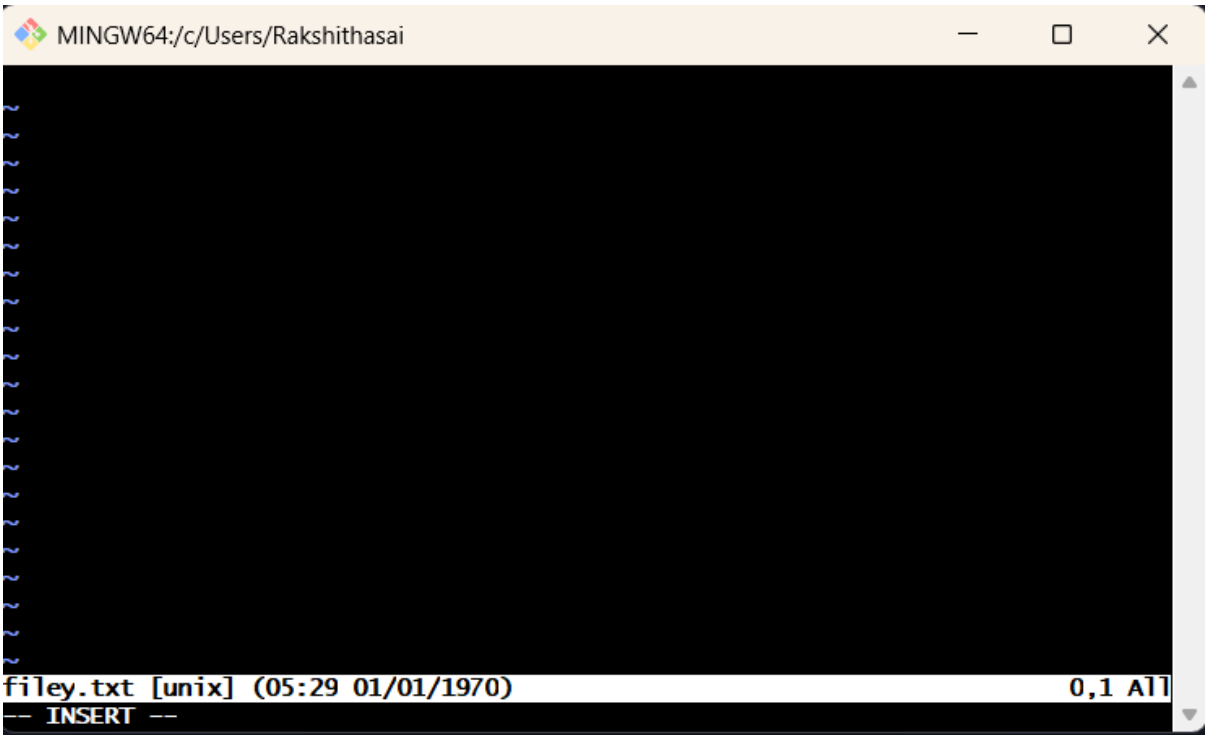
vi filename.txt → Opens file in Vi editor.

```
Rakshithasai@Rakshitha MINGW64 ~/OneDrive
$ vi filey.txt
```

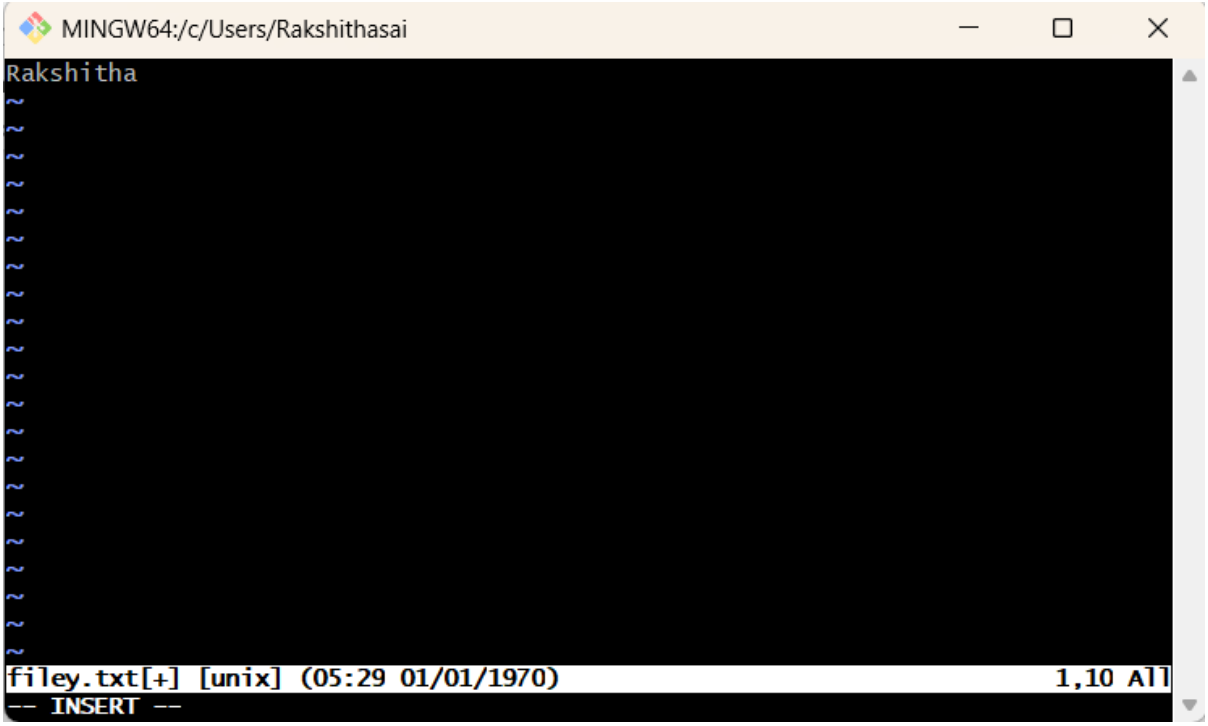


Insert Mode commands:

i → Insert before cursor

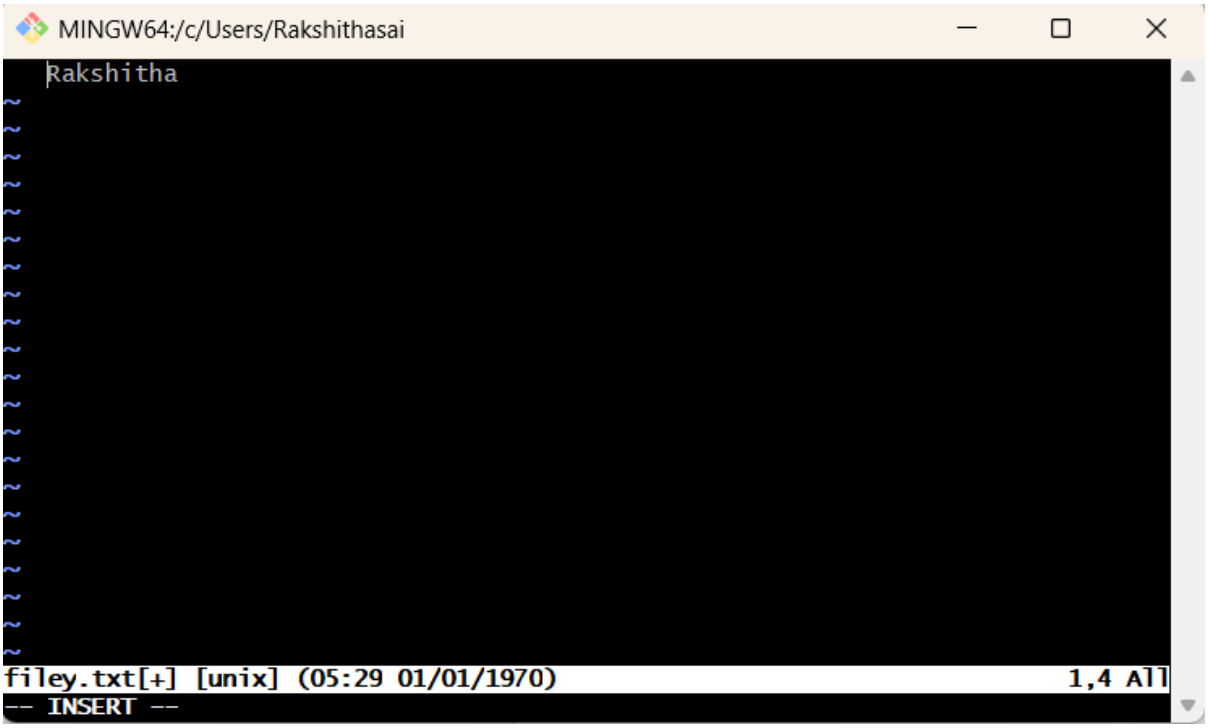


In the above image it is in insert mode.



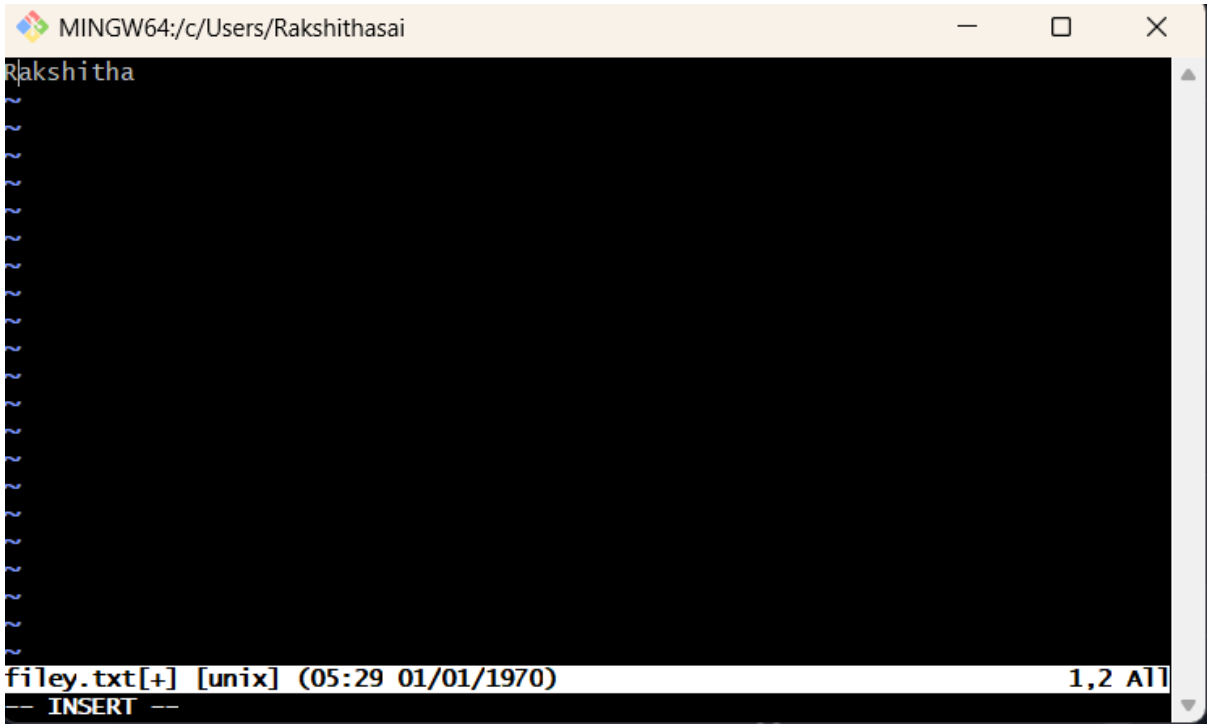
The insert mode allows to write the text in vi editor.

I → Insert at beginning of line



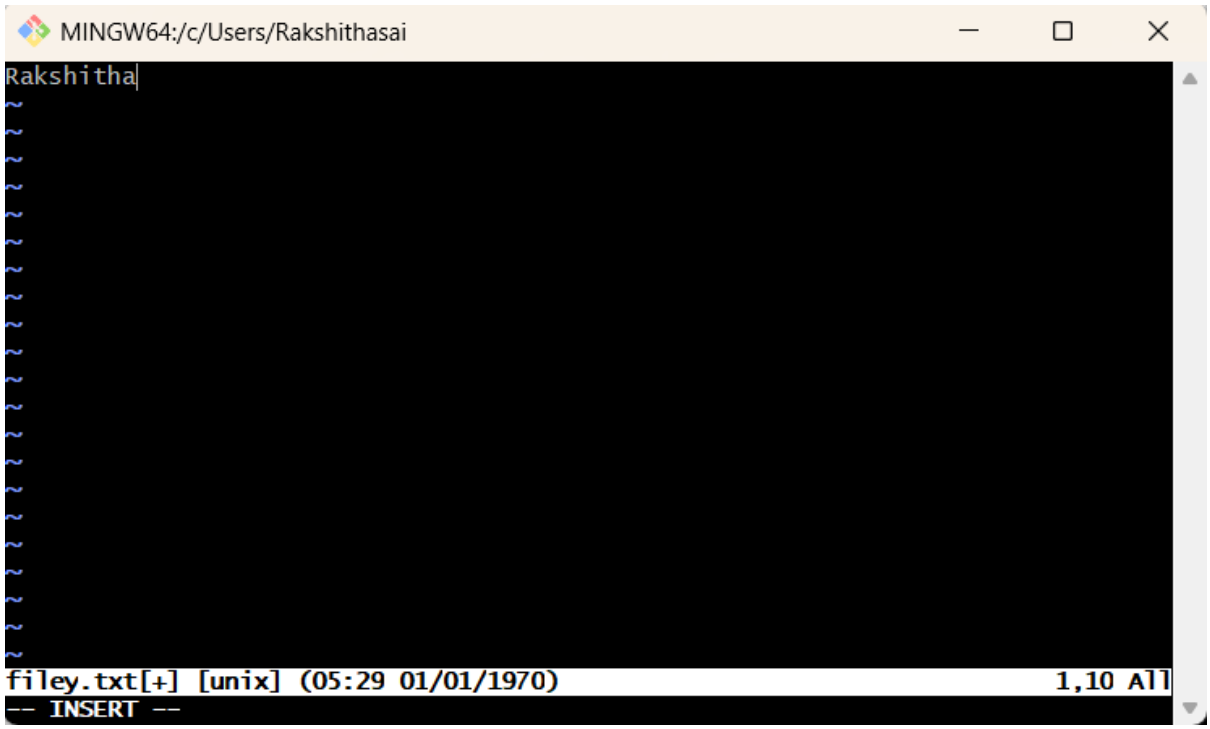
If your cursor is anywhere in the line (say on the word “Rakshitha”), press: “I”. The cursor jumps to the start of the line , and you’re now in insert mode.

a → Append after cursor



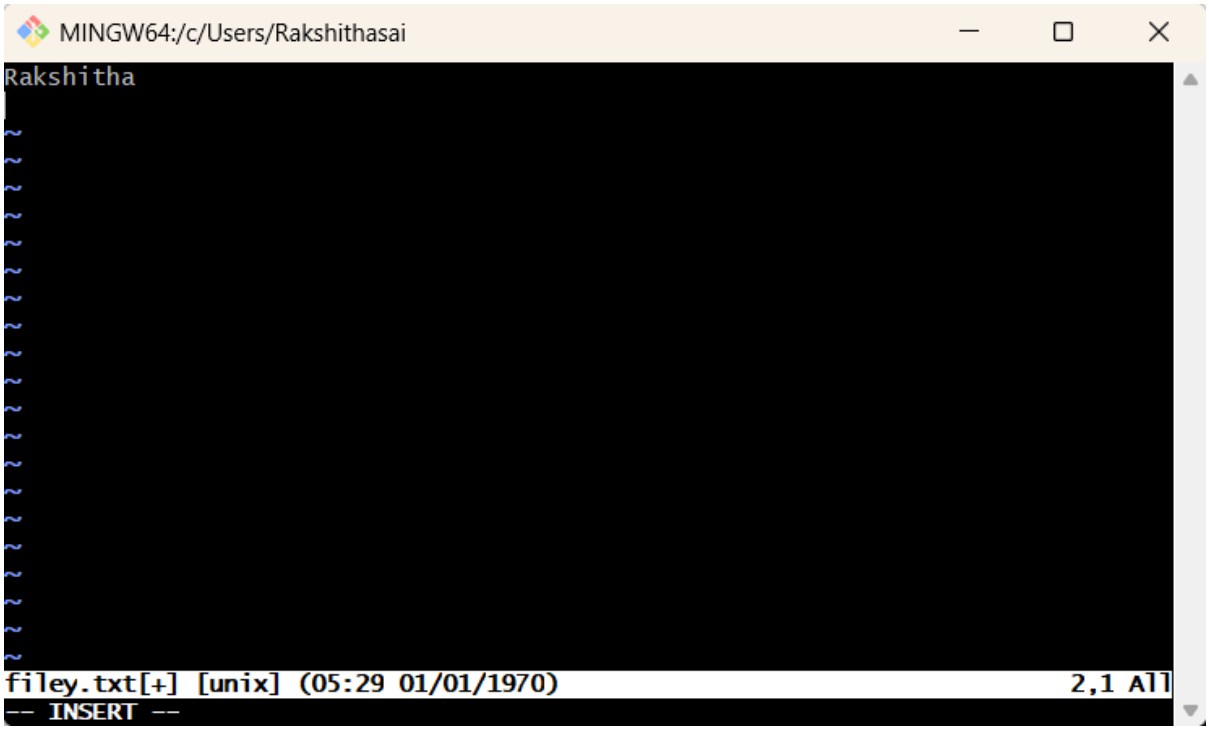
In the above image cursor moves to the right to append the text.

A → Append at end of line



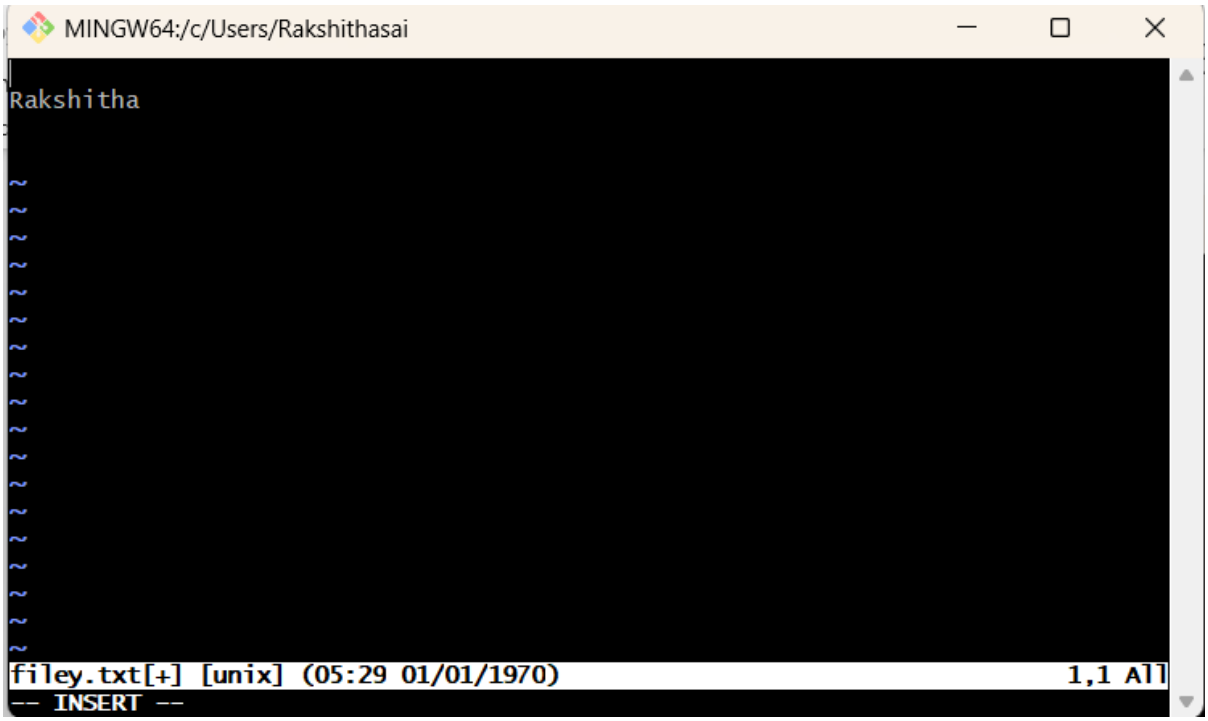
In the above image cursor moves to end of the line to append the text.

o → New line below cursor



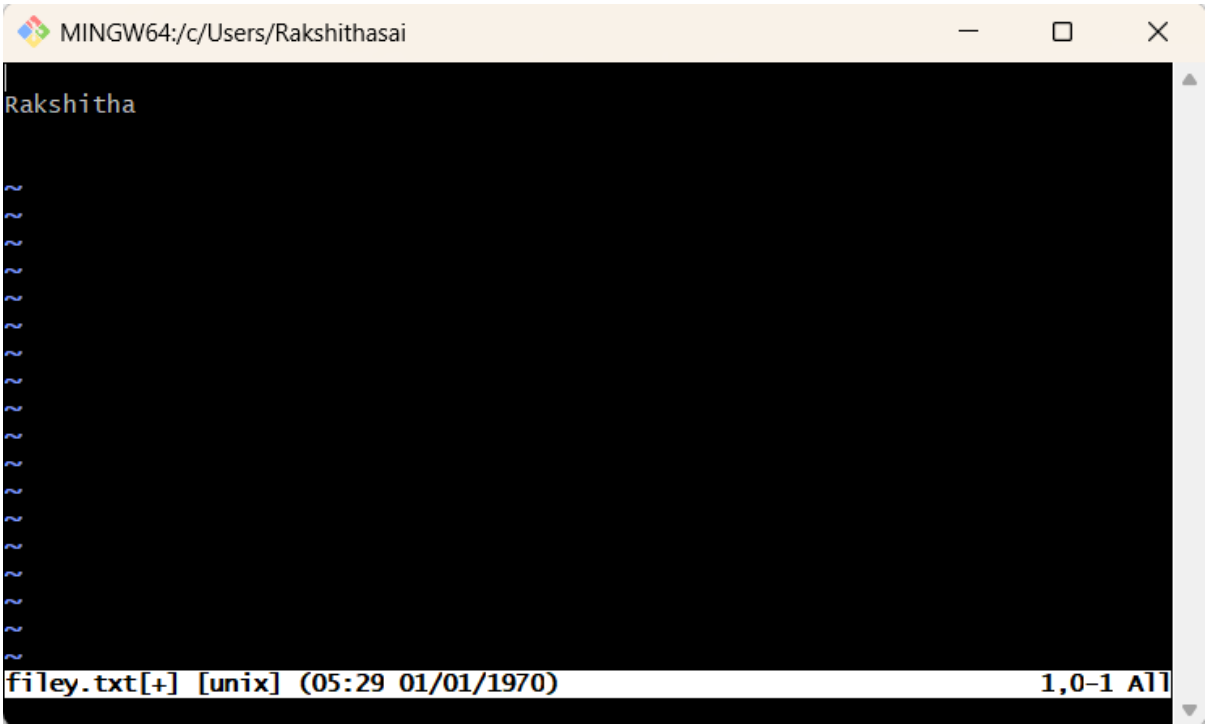
By using ”o” the cursor is moved to below new line.

O → New line above cursor



By using ”O” the cursor is moved to above new line.

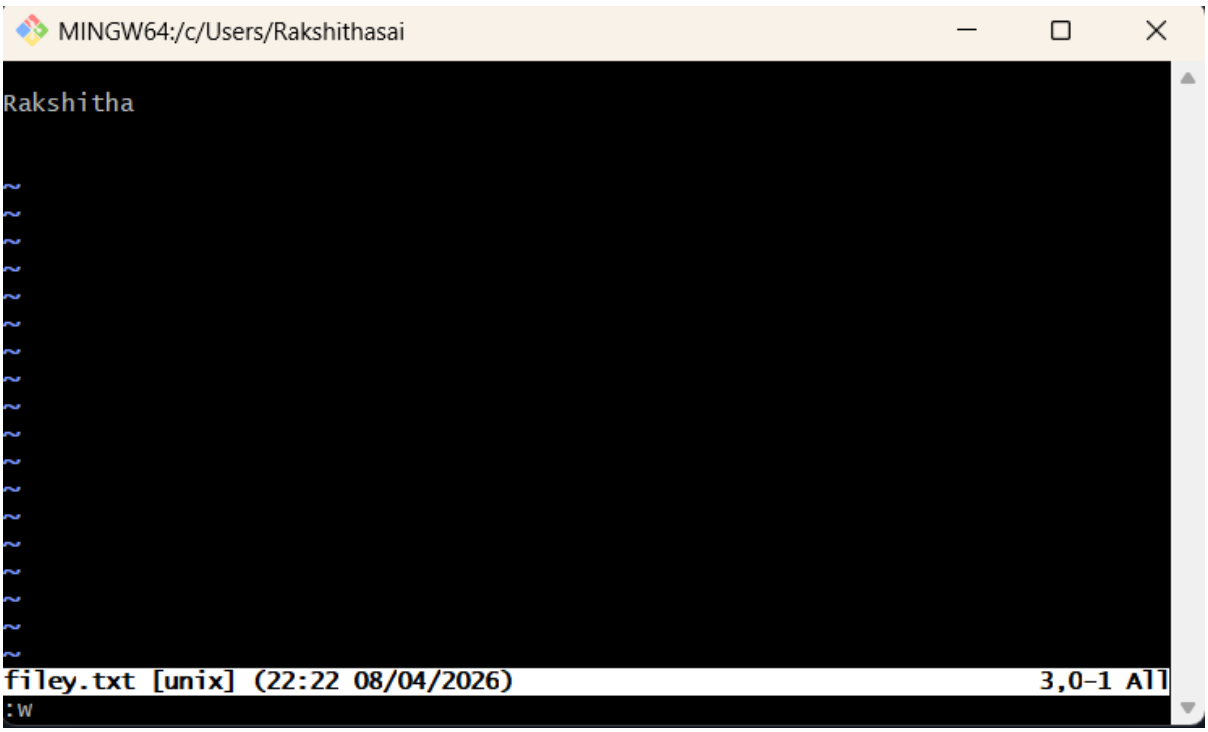
Esc → Exit insert mode



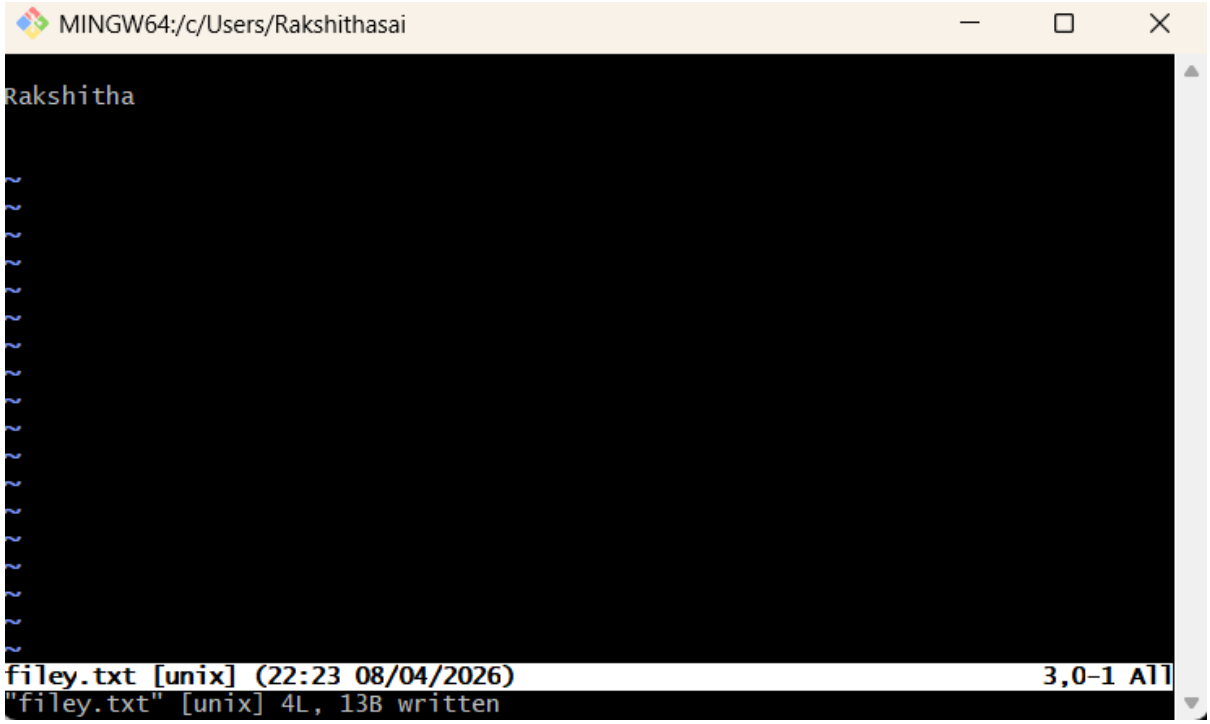
In the above image “Esc” exits insert mode.

Save & Quit in Vi

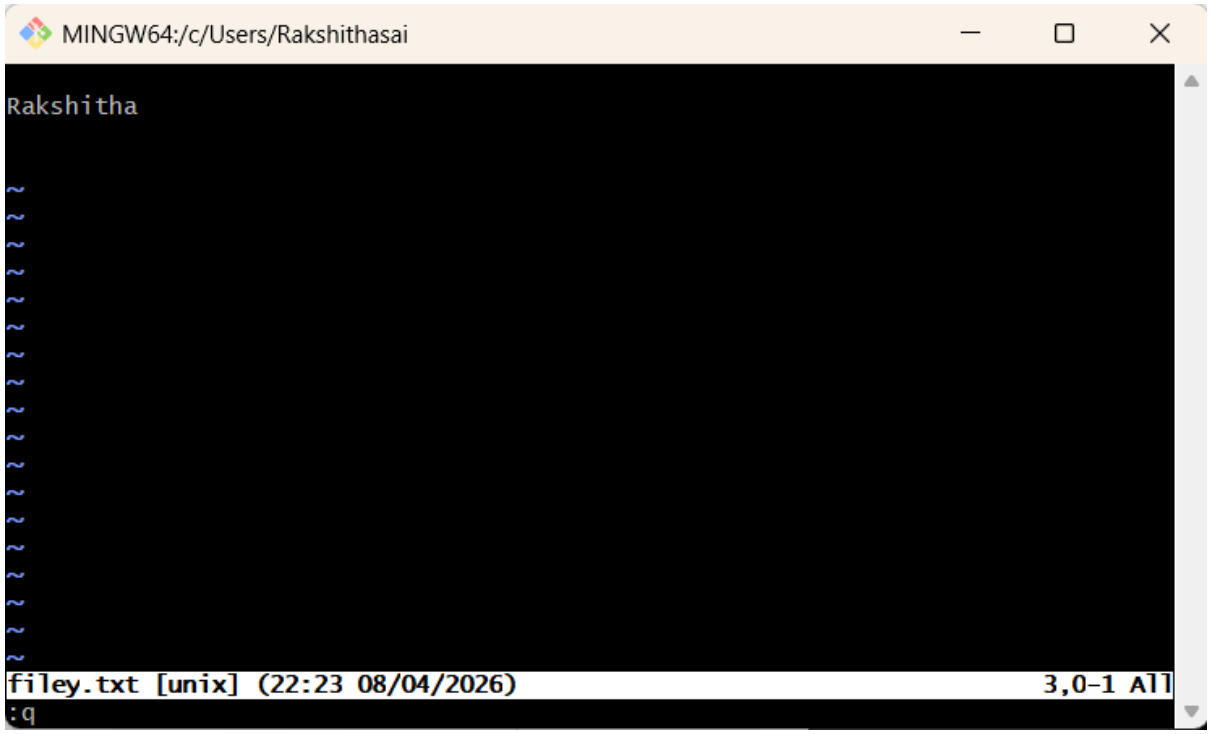
:w → Save file

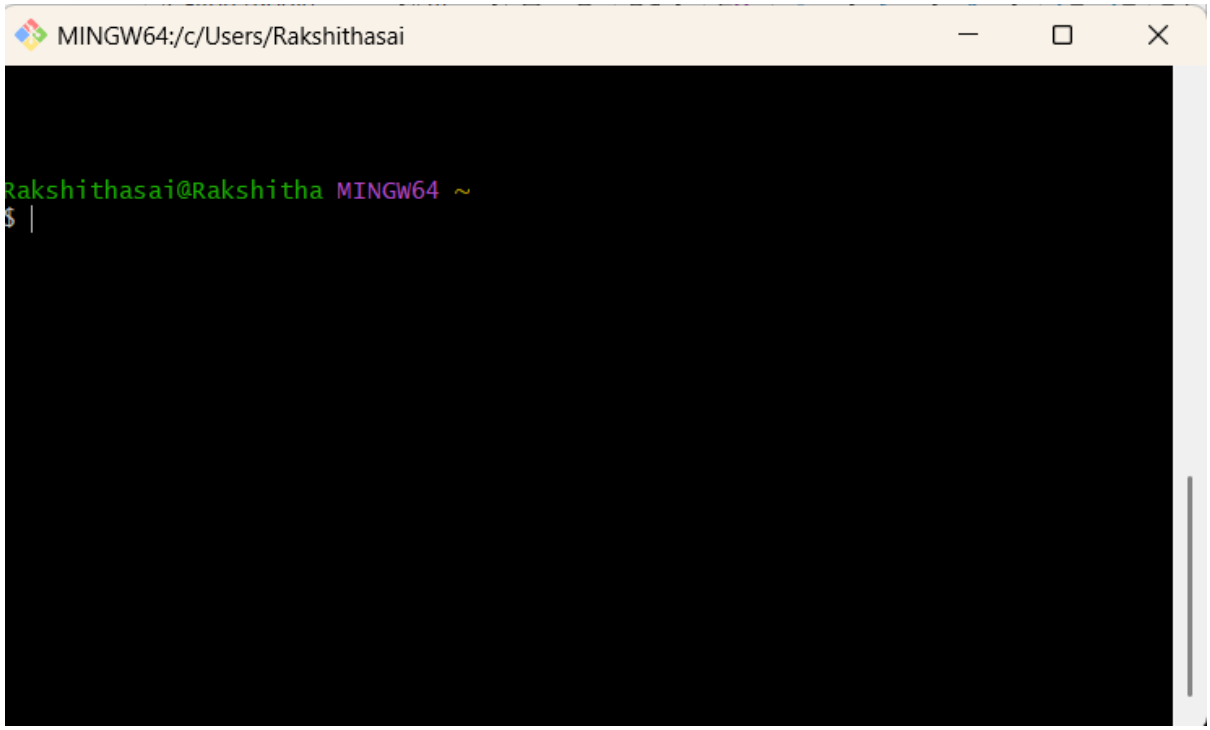


After exiting insert mode, It saves the file by using ”:w” command.



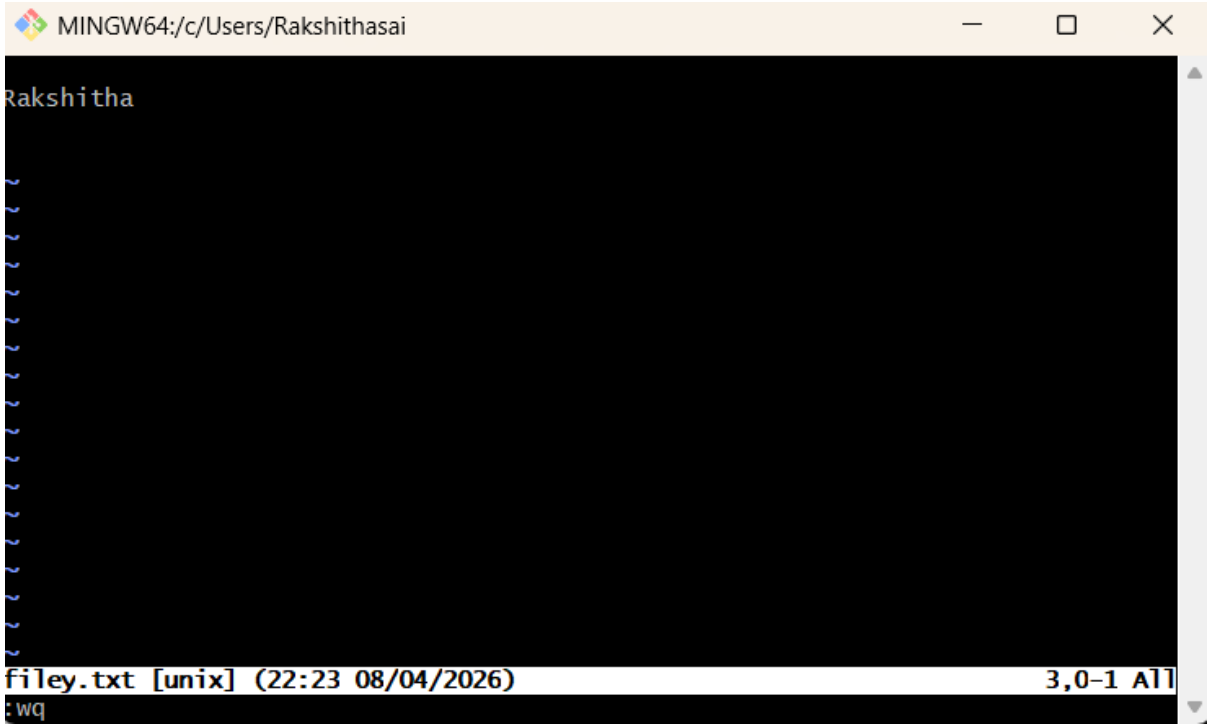
:q → Quit editor

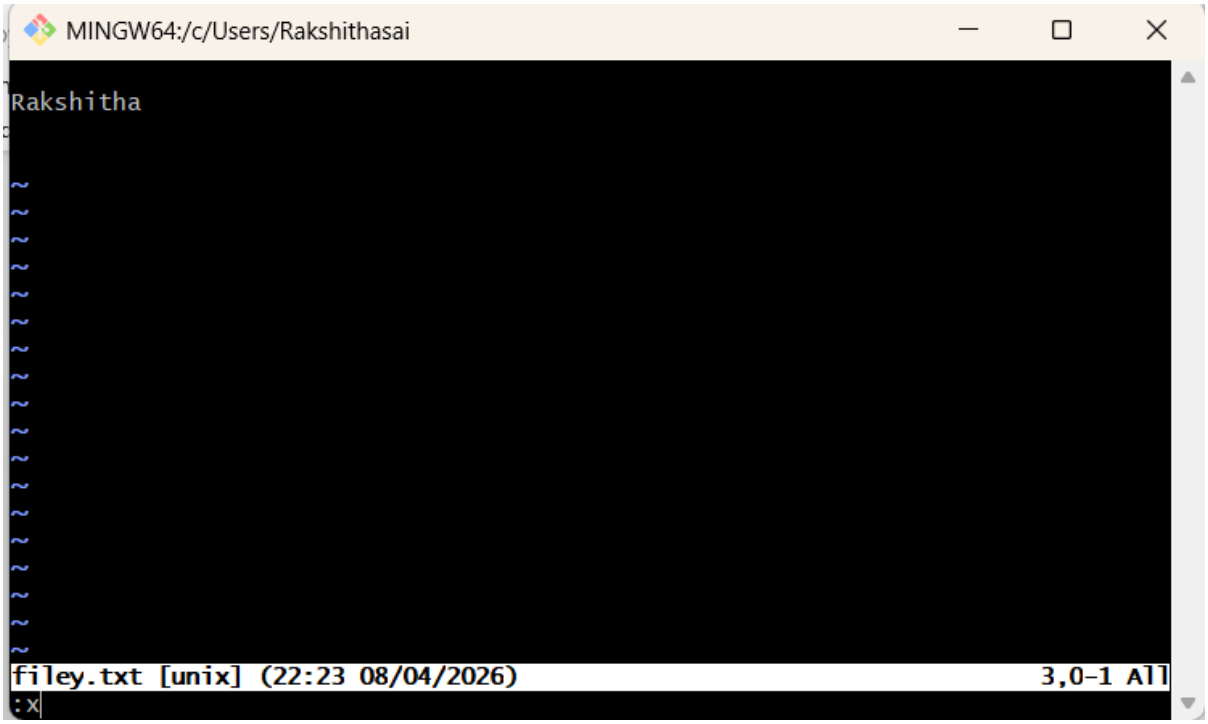


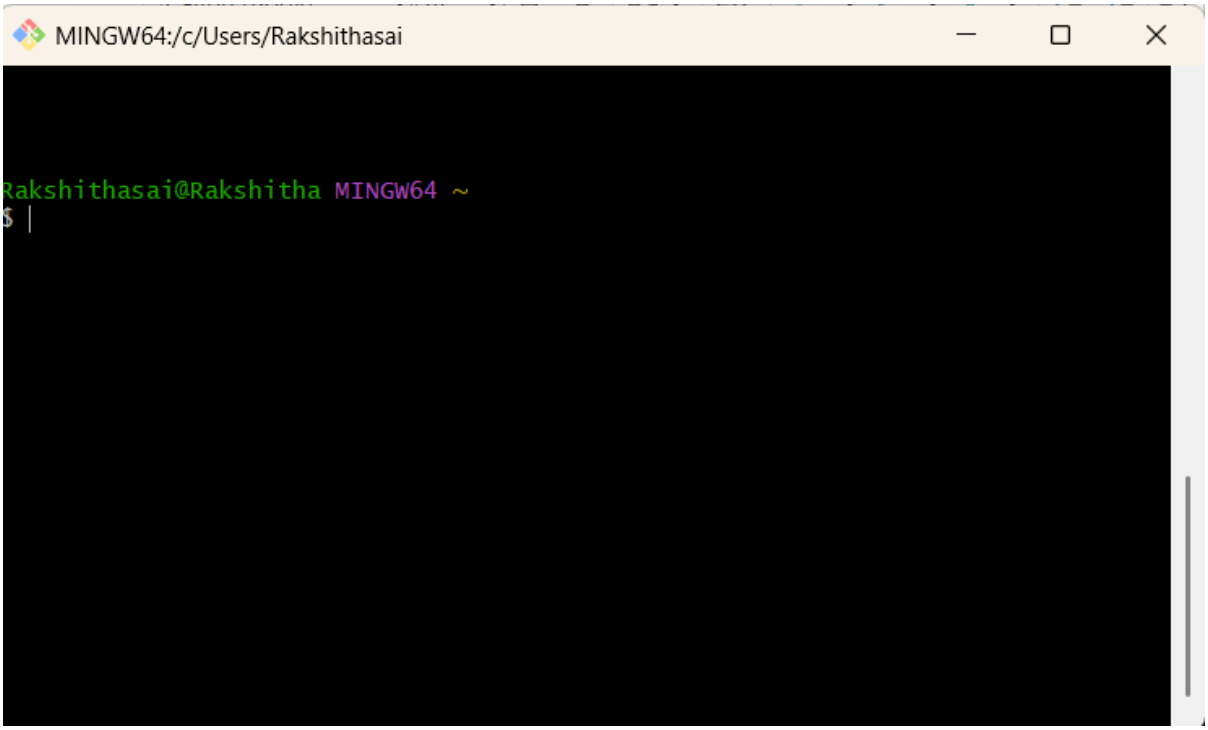


By using ":q" we quit vi editor to Git Bash.

:wq or :x → Save and quit

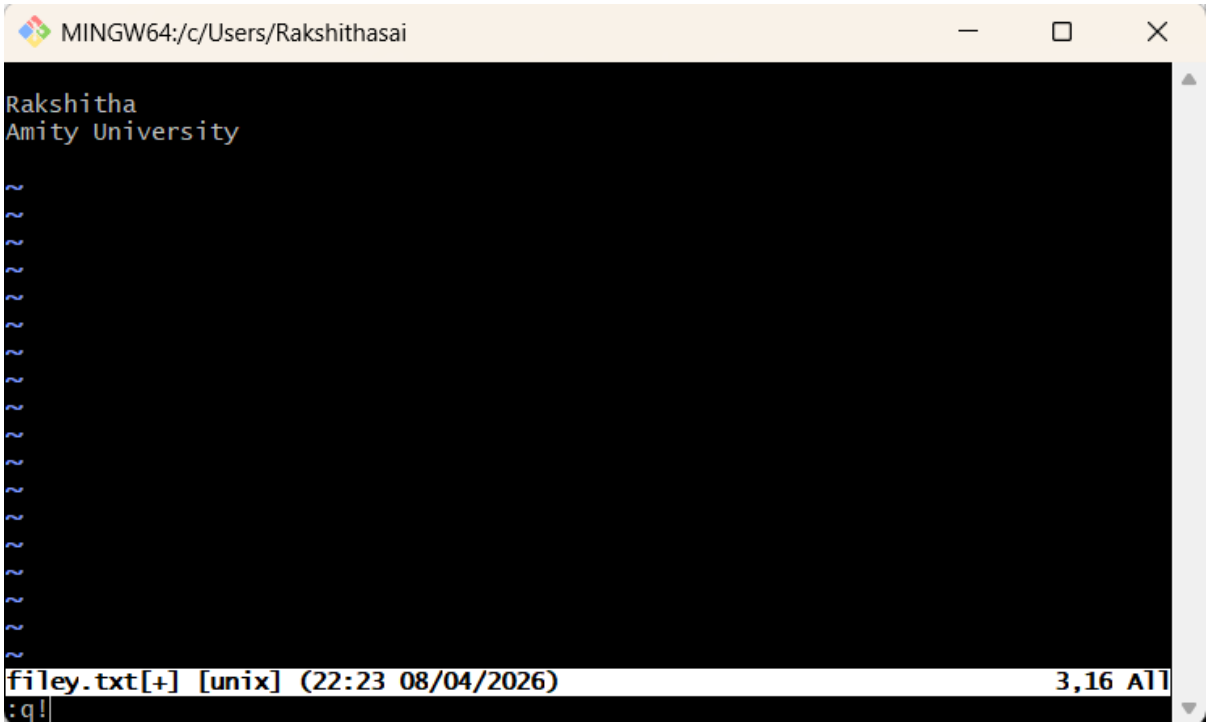




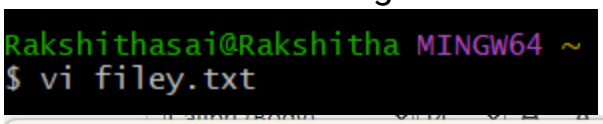


By using ":wq or :x" it saves and quit to the Git Bash.

:q! → Quit without saving

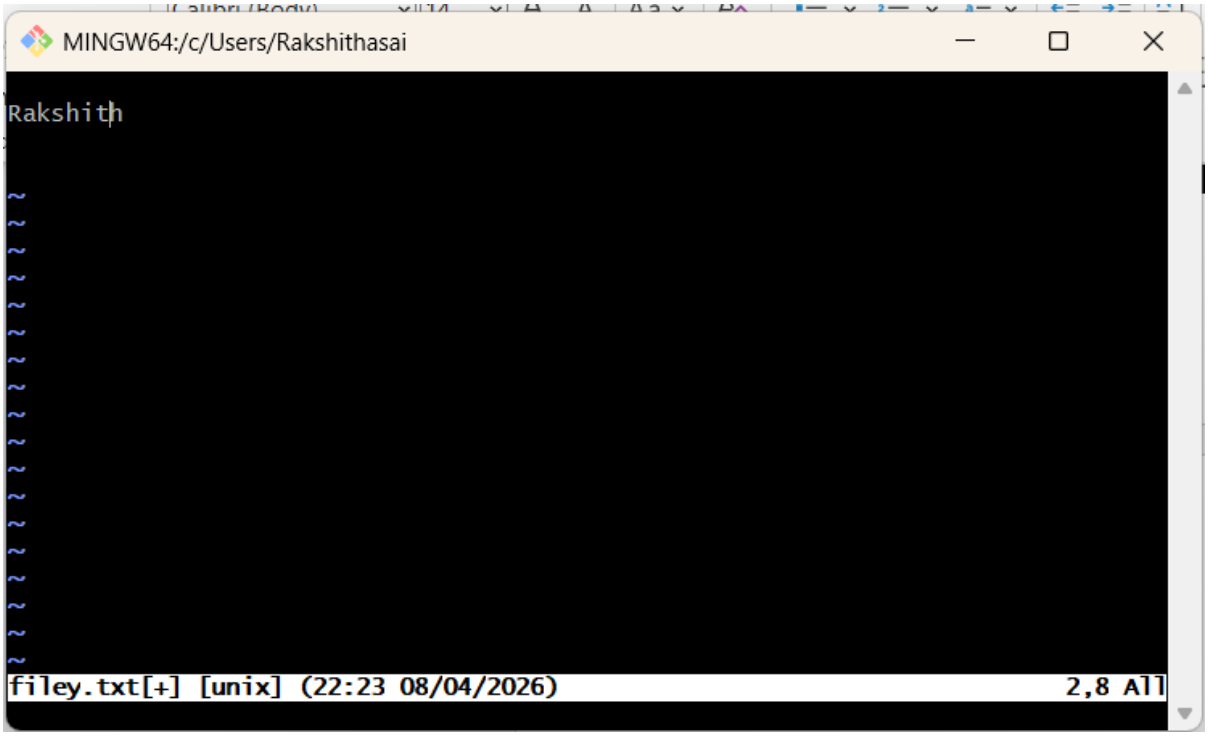


In the above txt “Rakshitha (new line) Amity University”, Amity University is not saved in the file because we used “:q!” command here it does not save the changes of the text and quit to the Git Bash.



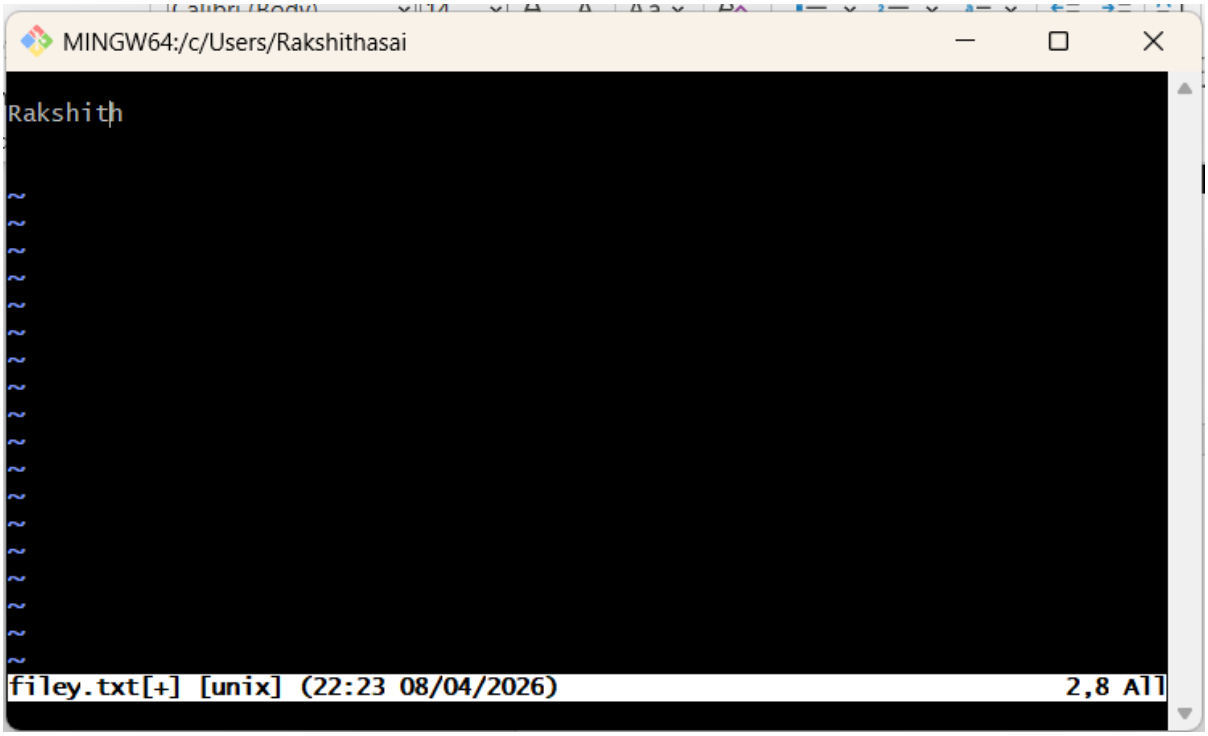
Delete Commands in Vi

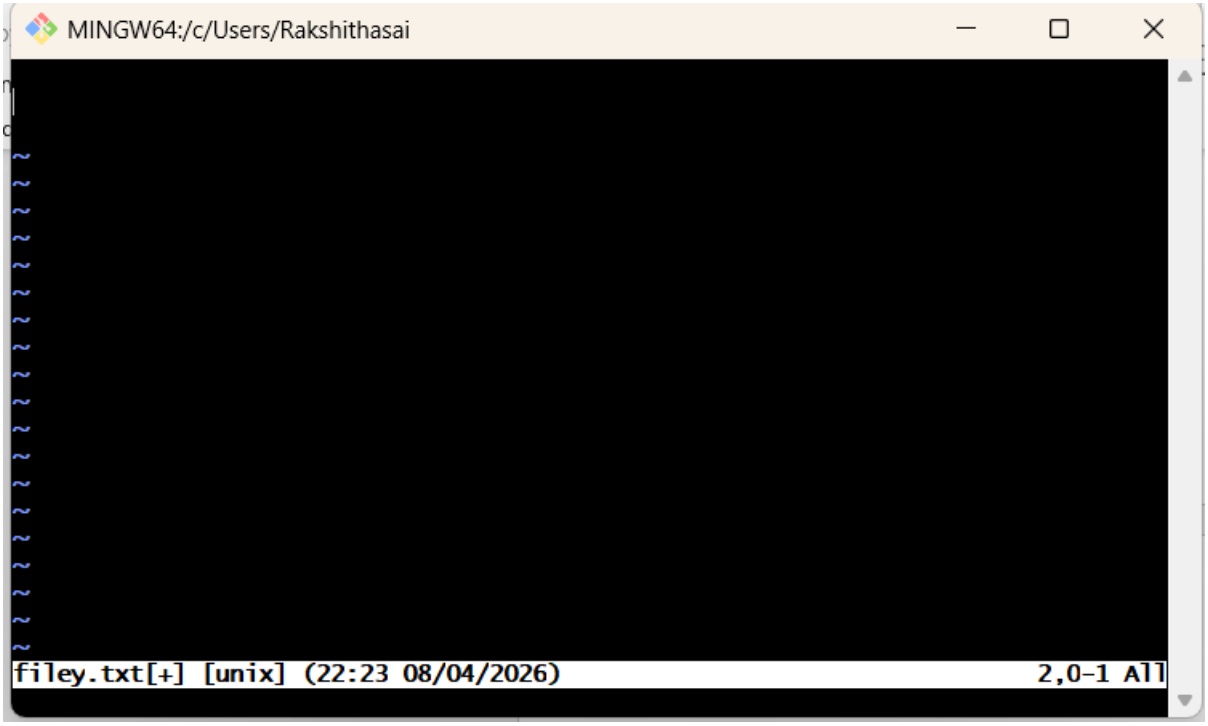
x → Delete character



It deletes a character before the cursor you can observe in the above image that in ”Rakshitha”, “a” is deleted.

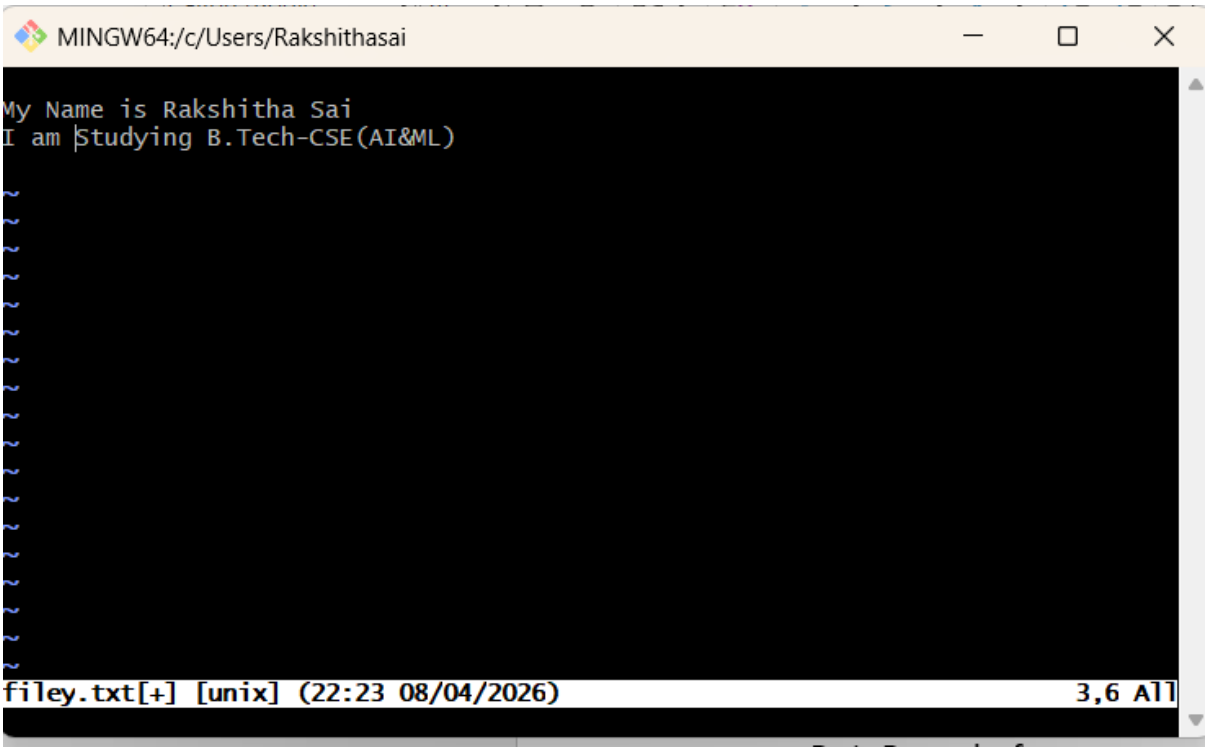
dd → Delete current line



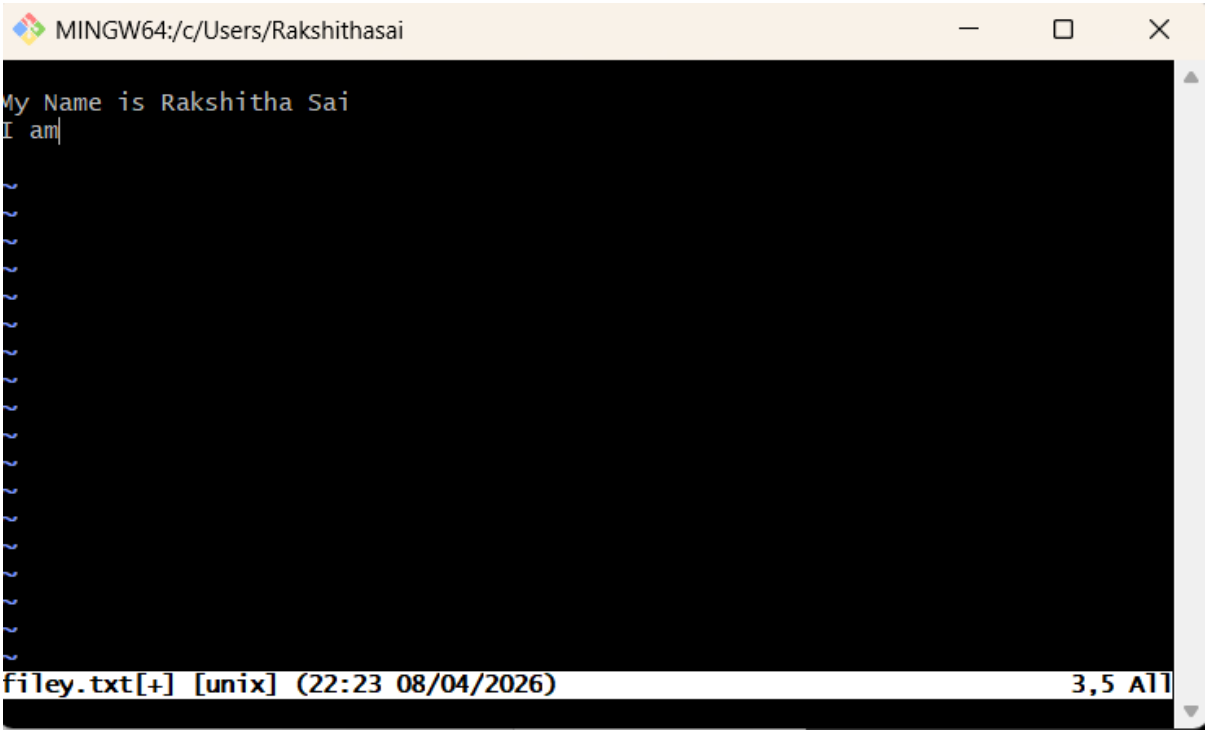


In the above text I have used “dd” command ,it deleted the line.

d$ → Delete from cursor to end of line

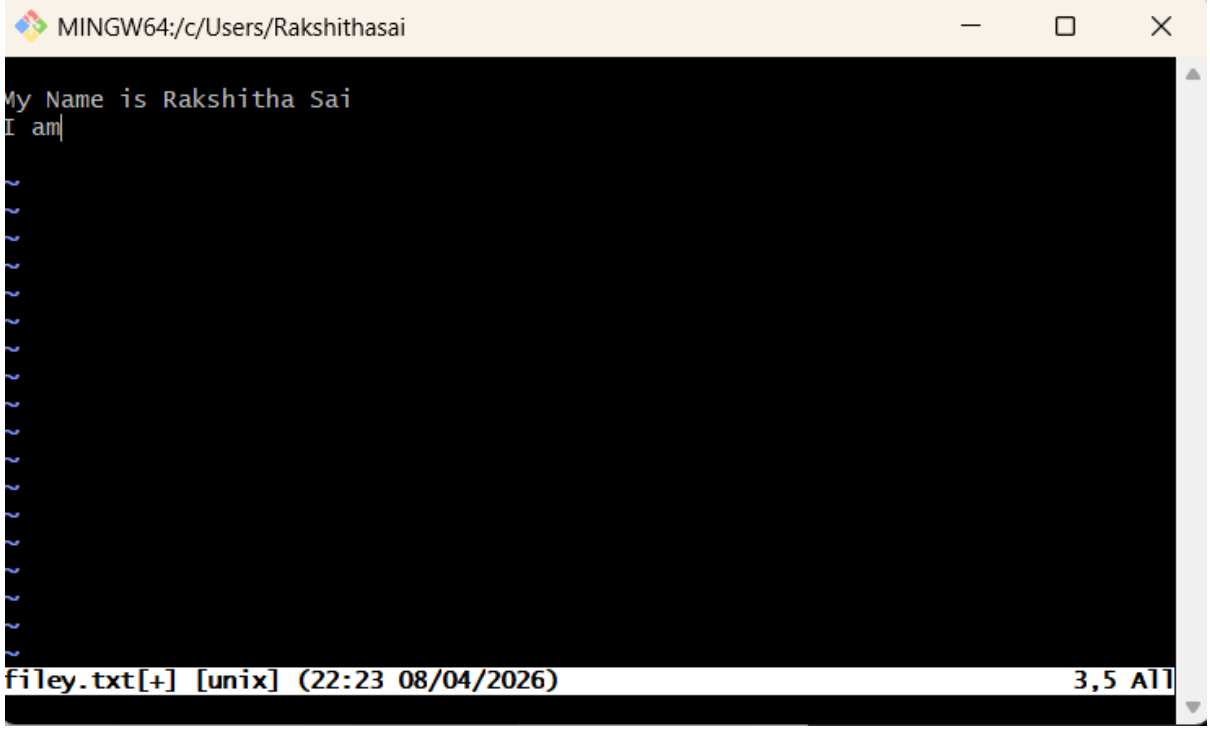


In the above image the cursor is present at second line.

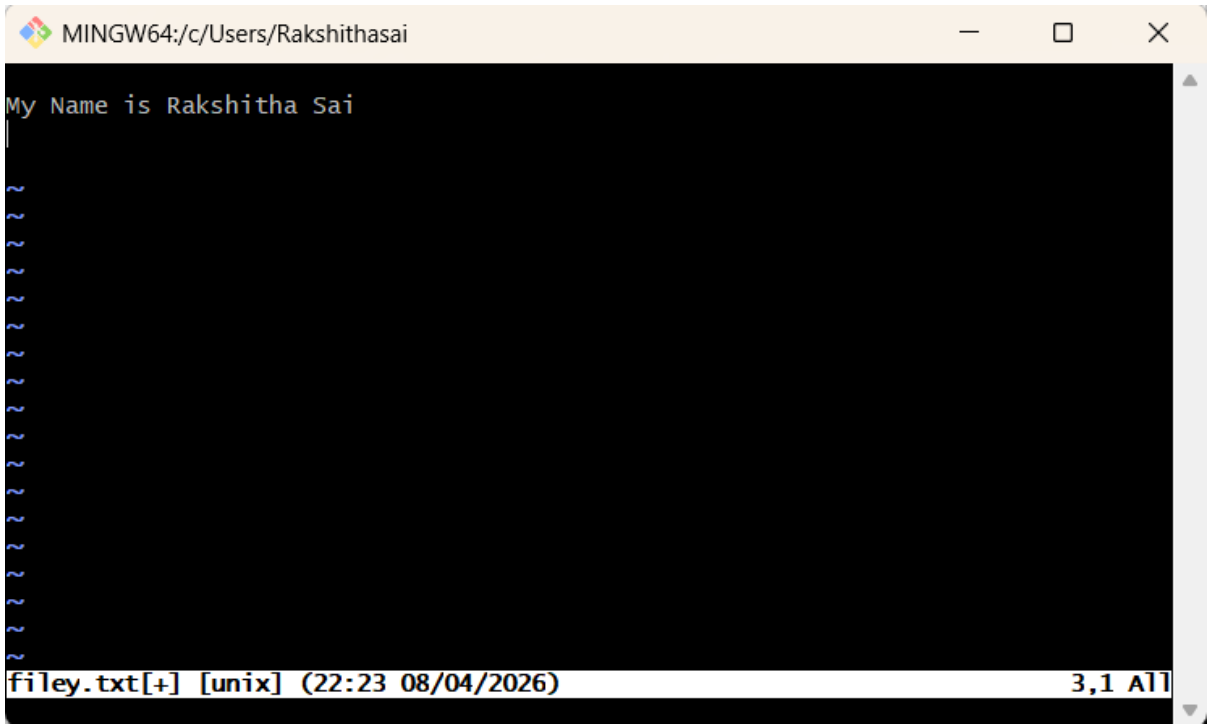


In the above line I have used “d$” command it deleted all the text from cursor to end of the line.

d^ → Delete from cursor to start of line



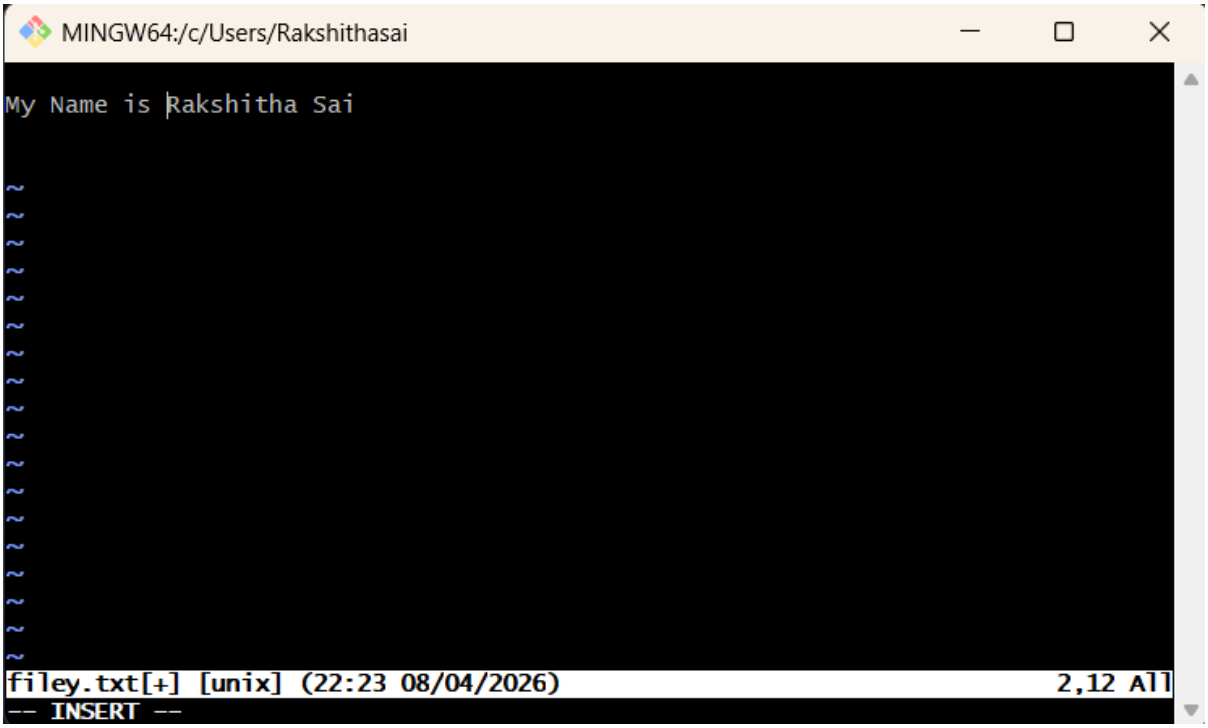
In the above image the cursor is in the second line.



We can observe that the “d^ ” command deleted all the text from coursor to start of the line.

Copy & Paste in Vi

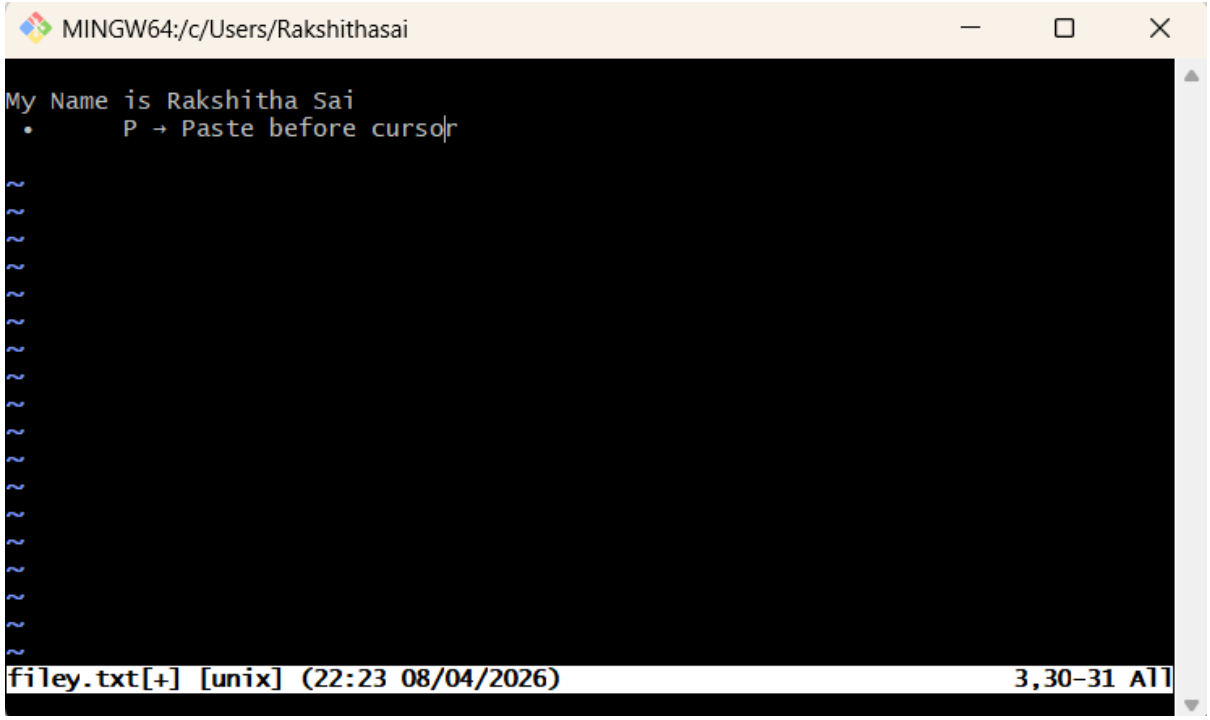
yy→ Yank (copy)



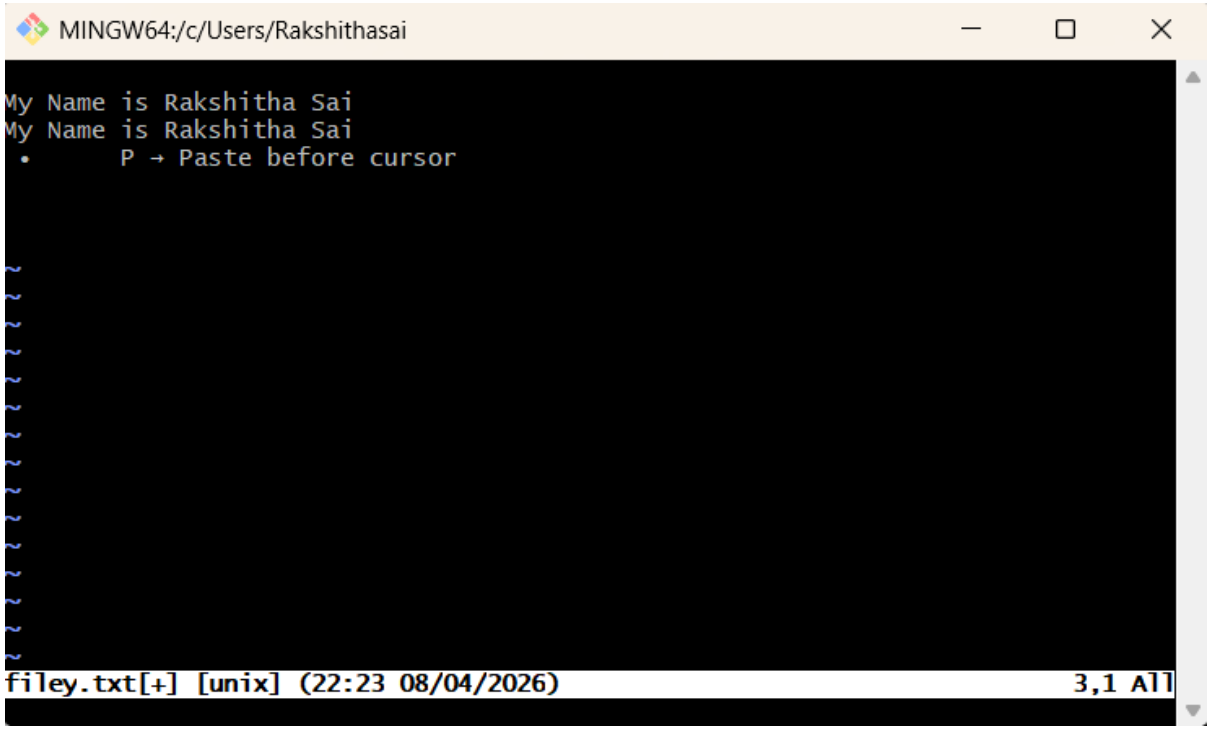
Move the cursor to the line you want to copy and Make sure you're in command mode.

I have used the command "yy" it does not display on vi editor,it only copies the line where the cursor is present.

p → Paste after cursor

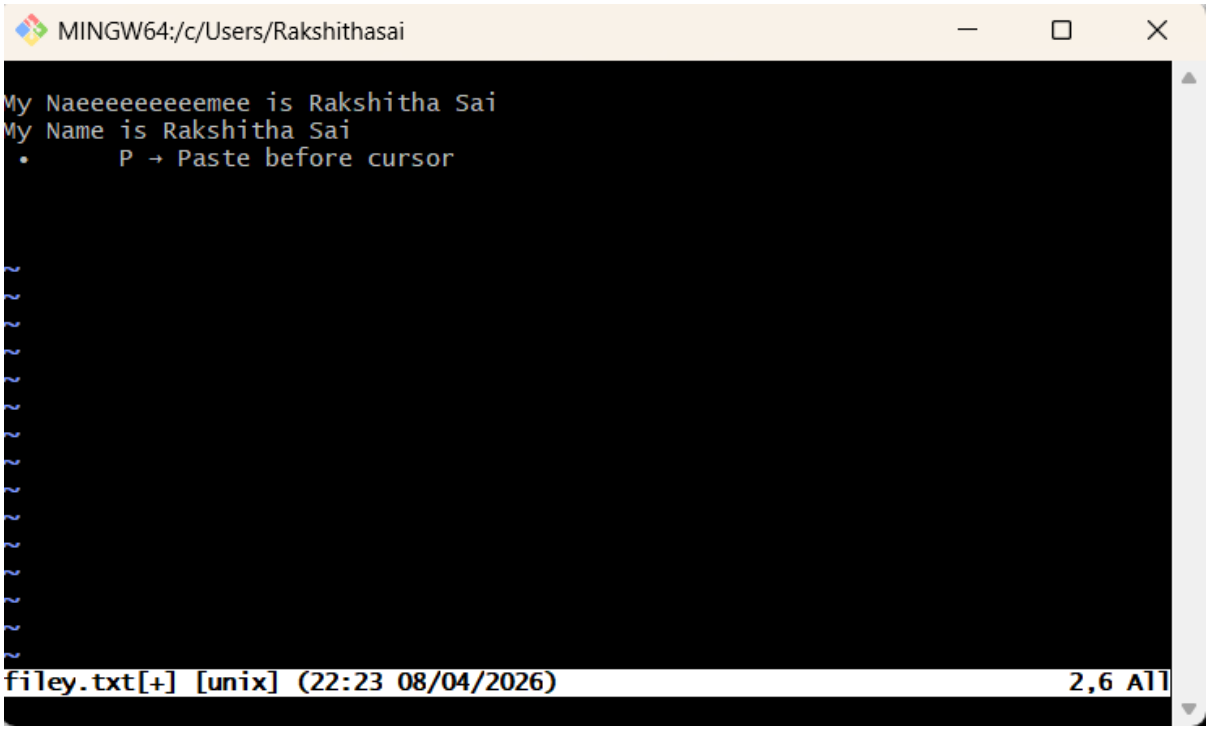


In the above image “p” command mentions” Paste before cursor”.



In the above image the copied line is pasted after the cursor.

P → Paste before cursor



In the above image I have placed the cursor before “e”, the more time I use “P” command the more “e” character is printed in the line before the cursor.